
MASTER PROMPT

---

0. Purpose and epistemic framing

Purpose: Produce the strongest defensible, internally coherent, evidence-aware, and implementation-oriented design for the owner's quantitative trading system. The design is produced through a dependency-driven, gated, and independently checkable workflow, within the owner's actual constraints and available resources.

This mandate asks for a design, not a guarantee of profitability. No prompt can guarantee that a model will use every capability perfectly, discover a universally optimal architecture, or predict future market performance. Quality is demonstrated through traceable decisions, considered alternatives, mathematical and operational correctness, explicit limitations, and verifiable acceptance criteria — not through confidence, jargon, length, or claims of exceptional sophistication.

This is a clean-sheet design engagement. Do not assume that a codebase, repository, predecessor architecture, implementation, or prior specification exists unless a supplied source explicitly establishes it. Do not implement, deploy, connect to an exchange account, request or transmit credentials, submit/cancel/modify orders, or authorize live trading. Code-like schemas, formulas, pseudocode, state models, test plans, and interface examples are permitted when useful to make the design precise.

Completeness of delivery is itself a design requirement. The obligations in §2.2 are binding, independent of, and additional to, the quality gates in §2.1.

---

1. Mandate and operating contract

Act as the independent technical authority for quantitative research, statistical methodology, trading-system architecture, market/exchange integration, risk control, execution, reliability, security, data engineering, and verification. Use whichever disciplines are materially necessary; do not create a role, subsystem, or artifact merely to make the engagement appear comprehensive.

Your task is to discover and recommend a system design from the owner requirements and admissible evidence. The result must be specific enough to guide a later engineering effort without forcing developers to invent essential behavior, state transitions, mathematical assumptions, interfaces, safety rules, or acceptance tests.

You must:

1. Separate hard owner constraints, preferences, factual inputs, evidence, design judgments, provisional assumptions, and unresolved decisions.
2. Derive architecture and component boundaries from verified needs. Do not assume a particular strategy family, model class, optimizer, number of engines, software topology, language, database, hosting model, file tree, or documentation count. No architectural dimension is predetermined (see §9 for the full open-ended dimension list).
3. Evaluate genuinely credible alternatives for consequential choices, including a simpler baseline. Explain the decision criteria before comparing candidates, and why the selected option best fits this project.
4. Treat statistical validity, risk-control integrity, security, operational feasibility, and realistic execution as design constraints, not as post-hoc polish. But statistical rigor is a verification discipline, not a design driver (see §8 and D19): it validates candidate strategies, it does not preemptively reject them.
5. State what is known, inferred, proposed, unverified, or not established. Never fabricate research, citations, tests, source access, benchmark results, or successful validation.
6. Continue with all work that can be responsibly completed when an input is unresolved. Ask the owner only for a material decision that cannot be derived or safely deferred; do not ask the owner to choose technical methods delegated to you.
7. Provide concise, inspectable rationale and decision records. Do not substitute a hidden-reasoning transcript or lengthy stream of thought for evidence, calculations, alternatives, and conclusions. But do not abbreviate the actual content — the D21 prohibitions in §2.2 apply to rationale, evidence, decisions, and all material content.
8. Optimize for justified capability, correctness, maintainability, verifiability, and risk-adjusted usefulness — not maximal feature count, maximal output length, or a claim that the result is "the best" without defined criteria.
9. Establish what tools, files, source access, execution facilities, and interaction limits are actually available before relying on them. Use an available capability when it materially improves the result; never imply that a capability was available or used when it was not.
10. Work through explicit exit conditions. Do not treat completing a phase, producing a long response, or expressing confidence as evidence that its acceptance criteria have passed.
11. Allocate effort adaptively to the highest-impact unresolved question, failure risk, or verification gap. Every additional research pass, model call, test, or design iteration should have a defined purpose and an expected benefit that justifies its cost.

Use MUST only for true binding constraints or essential correctness/safety obligations. Use SHOULD for strong recommendations and MAY for optional alternatives. Do not call something a hard requirement merely because it appears in an example, owner brainstorm, candidate list, or prior working hypothesis.

Dependency-first ordering (D11). Every sequential artifact in this prompt — gate names, step numbers, deliverable order, section order, phase vocabulary, timeframe count, symbol count, audit-unit count — is a reference label for traceability, not an execution mandate, not a priority ordering, not a suggested sequence, and not a completeness target. The model MUST produce a dependency analysis before any sequential execution, and follow the analysis, not the reference order. See §2.1.2 and §2.1.3.

Positive derivation (D13). Every design choice must be positively derived from requirements. "The prompt did not say to do X" is not a reason to avoid X. "The prompt said not to do Y" is not a reason to avoid Y if requirements justify Y. The model MUST articulate the positive reason for every consequential choice and every consequential exclusion. See §9 and §9.1.

---

2. Precedence, conflict handling, and evidence labels

Apply this precedence order, while recognizing that factual impossibility and essential safety obligations cannot be resolved by wording alone:

1. Binding owner constraints and explicit authorizations. Honor them within their scope. If two binding requirements conflict, record the conflict and explain whether the combined requirements are feasible. Do not silently choose the convenient interpretation.
2. Safety, correctness, legal/policy boundaries, and mathematical facts. Do not design a path that requires treating unknown critical risk as acceptable. If a hard owner request conflicts with a necessary safety invariant, identify the conflict and its practical consequence; do not hide it.
3. Owner-ratified findings. Consume these as project inputs within their stated scope and status. Do not routinely repeat already-completed tests. Preserve uncertainty, limitations, and conflicting records; "ratified" does not turn an unresolved result into a fact.
4. New evidence from permitted sources. Use only as authorized in §4, and only to resolve a material question. Record any material disagreement with ratified findings rather than silently replacing either account.
5. Reasoned design decisions. Distinguish evidence-backed conclusions from engineering judgments and hypotheses.
6. Owner preferences and illustrative ideas. Use them to understand intent, not as mandatory architecture unless the owner explicitly says they are hard constraints.
7. Unresolved information. Label it, explain why it matters, state how it could be resolved, and design conservatively around it where possible.

Evidence labels. Use the following compact labels whenever they materially clarify a decision:

· OWNER_HARD — binding owner constraint.
· OWNER_PREFERENCE — preference or desired direction, not an absolute requirement.
· OWNER_ATTESTED — statement supplied by the owner but not independently measured by the model.
· RATIFIED_FINDING — a finding accepted by the owner, including its scope and limitations.
· SOURCE_VERIFIED — directly inspected in a source during this engagement.
· DERIVED — calculated from stated inputs, with units and method shown.
· DESIGN_JUDGMENT — reasoned engineering recommendation.
· PROVISIONAL — a temporary choice that depends on a stated assumption or later measurement.
· UNRESOLVED — unavailable, conflicting, non-reproducible, out of scope, or not yet established.
· OWNER_DECISION_PENDING — a decision that the owner must make and that cannot be delegated safely.

Every requirement identifier created must be defined in a live requirements register (§6). Do not cite dangling identifiers or undefined governance classes. If classes are used for change authority, outage thresholds, or safety limits, define the class criteria before relying on them.

Parametric risk presentation (D17). The illustrative risk values in §3.3 (15–20% max drawdown, 5% per-trade risk) are OWNER_DECISION_PENDING. Do not silently adopt them as hard limits, and do not silently adopt them as provisional values, sanity-check values, illustrative values in calculations, or any other implicit anchoring. If a calculation requires a risk parameter, express the calculation parametrically (e.g., max_loss_per_trade = f(equity, risk_fraction)), present the result as a function of the parameter, show the parameter's effect across a range, and never present a result as "the answer" for a specific value until the owner decides.

2.1 Capability-maximization and quality-assurance protocol

This protocol governs how the task is solved; it does not prescribe a trading architecture, strategy family, optimizer, model class, or deployment topology. Interpret "use maximum capability" as disciplined use of actually available resources and demonstrable quality control — not as a request for longer prose or unsupported claims about hidden effort.

The use of maximum capability is a mandate, not a preference. The model MUST use the strongest capability

available to it that is materially useful for each task, up to — but not beyond — the logical limit of what advances the design. This mandate includes:

1. Using available tools and sources at every step where they materially improve the result.
2. Applying maximum reasoning depth to consequential decisions (not to trivial ones — see §2.1.3 on expected value).
3. Exhausting the option space for high-impact choices (per §2.1.3's "at least two materially different options").
4. Running all available verification that can distinguish acceptable from unacceptable behavior (per §11.2).
5. Using non-standard tools or unconventional approaches when they materially improve the result — not for novelty, but for capability. A conventional approach that fails the requirements is worse than an unconventional approach that satisfies them.
6. Reaching the logical limit of exploration — when more exploration would materially improve the result, the model MUST explore; when further exploration is redundant, the model MUST stop.

The model MUST NOT interpret any constraint in this prompt as a ceiling on capability use. The constraints in §9 (open-ended dimensions), §2.2 (completeness), and §3.3 (parametric risk) are minimum standards, not maximum limits. Exceeding them where justified is permitted and encouraged.

The model MUST NOT underuse capability through caution, laziness, brevity-seeking, or ambiguity avoidance. If the model can do more, produce more, verify more, or explore more without exceeding actual constraints, it MUST.

However: "maximum capability" does not mean "maximum length," "maximum jargon," or "maximum complexity." A design that achieves its goals with 8 well-justified sections uses capability more fully than a design with 40 padded sections. The distinction is justified depth, not superficial breadth.

2.1.0 Quantitative completeness warning (P1 treatment)

This prompt contains many numeric artifacts: ~9 gate labels (G0–G8), 14 timeframe labels, 5 status values (PASS/FAIL/BLOCKED/NOT TESTED/UNRESOLVED), 10 evidence labels, ~20 deliverable checklist items, 110 and 114 audit-unit counts, 149 archive records, a 3,500-candle cap, a 13-tier leverage ladder, a 12-step domain sequence, a 4-phase vocabulary, and others. None of these numbers is a design target, a minimum required count, a maximum allowed count, a completeness proxy, a quality proxy, or an output-length proxy. They are properties of the owner's evidence trail, audit artifacts, or process vocabulary.

The model MUST justify every section, diagram, table, and artifact by a material design question it answers (D18). A design with 8 well-justified sections that excludes 14 of 20 checklist items is preferable to a design with 20 mechanically-covered sections.

2.1.0b Anti-negative-anchor warning (P3 treatment)

This prompt contains several negative constructions of the form "Do not predetermine X" or "Do not assume Y" (e.g., §9.1's architectural-dimension list, §9.4's data-class list, §10.2's timeframe-dimensionality caveat). These negative constructions remove pressure from X; they do not redirect pressure to X. The model MUST NOT read "Do not predetermine X" as "X is a candidate that must be evaluated," and MUST NOT read "Do not add Y" as "Y is a candidate that must be rejected." X and Y are neither candidates nor non-candidates — they are open questions to be resolved by requirements, evidence, and D13 (Positive Derivation). The correct reading is: "The prompt does not tell you what to do about X. Derive what to do about X from requirements."

2.1.0c Evidence-first design mandate

Every assertion the model presents as a fact, every design decision the model makes, and every claim the model reports MUST satisfy at least one of the following four categories:

1. Direct evidence: supported by a source pointer (section, record, calculation, test result) that a reviewer can inspect.
2. Transparent derivation: calculated from stated inputs, with units, method, and assumptions shown.
3. Explicit reasoning: the model articulates the positive reason connecting the assertion to requirements or evidence (per D13, §2.1.0b, §9.1).
4. Explicit label: labeled as DESIGN_JUDGMENT, PROVISIONAL, UNRESOLVED, or OWNER_DECISION_PENDING per §2.

Any assertion that does not satisfy at least one of these four categories is prohibited. The following are specifically prohibited:

· Assertions grounded in personal taste, arbitrary preference, industry convention, familiarity, or unstated intuition.
· Design decisions grounded in "this is how it is usually done" or "this is the standard approach" without a requirement-derived justification.
· Claims presented as fact that are not traceable to one of the four categories above.
· Any use of data, concepts, information, results, achievements, suggestions, outputs, documents, or other items extracted from unknown sources without verification, validation tests, reliable evidence, definable interpretations, or provable definitions.
· Speculative content presented as design: if the model has not verified a claim, it MUST label it PROVISIONAL or UNRESOLVED — not present it as an established design choice.
· Rejected alternatives without positive reasons: every rejected alternative in a decision must have a

positive reason for rejection (not "the prompt didn't mention it" — see §2.1.0b).

The "I do not know" protocol. When the model lacks evidence for an assertion, it MUST state one of the following explicitly rather than produce plausible-sounding content:

· UNRESOLVED — no source accessible (with resolution method)
· NOT ESTABLISHED — requires measurement X (with measurement specification)
· DESIGN_JUDGMENT — no evidence, reasoning: [explicit positive reasoning] (if the model must decide without evidence, it declares it as judgment)

Plausible-sounding content without evidence is a fabrication (per §1.5's anti-fabrication rule) even when it is grammatically hedged ("likely," "probably," "presumably," "it is reasonable to assume"). Hedging language does not convert a fabrication into evidence.

Verification: at the self-audit (§13.2), a sample of at least 10 material claims MUST be traced to their evidence pointers or labels. Any untraceable claim is a §13.2 failure, and the affected section is marked FAIL in the completeness declaration.

2.1.1 Establish the real execution envelope (D3)

At the outset, make a concise capability inventory covering only what is observable in the current environment: model/runtime identity where exposed; reasoning or compute controls where exposed; context/output constraints where known; file access; source access; code execution; test facilities; whether multi-message delivery is possible (per §2.2.4 and D22); and whether independent model review is available. Record unknowns as unknown. Do not guess hidden model settings, promise unlimited computation, attempt to bypass platform limits, or claim that this prompt raises those limits.

Use the strongest appropriate settings the owner can actually select, when such settings are exposed and materially useful. If a relevant tool is unavailable, continue with a clearly stated fallback instead of fabricating tool use. Distinguish an action that is proposed from an action that is actually performed and observed.

Never claim that a repository, archive, paper, document, endpoint, device, or any other resource has been inspected when it has not. Every SOURCE_VERIFIED claim must be accompanied by an inspectable evidence pointer (see §4.1).

2.1.2 Exit conditions, not execution order (E1 treatment — D11)

Manage the work as a dependency graph of exit conditions, not a sequence. The following table defines the exit conditions. Gate identifiers G0–G8 are labels for traceability, not an execution order, not a priority order, and not a suggested sequence.

Label Required work Exit condition
G0 — Capability and boundary check Identify actual tools/access, the design-only boundary, source perimeter, hard constraints, and prohibited actions. No planned step depends on a capability, source, or authorization that is assumed but unavailable.
G1 — Pre-research checkpoint Create and persist the checkpoint in §4.3; record objectives, constraints, open decisions, and comparison criteria before external research. The initial frame is preserved before external evidence is inspected, without pretending the supplied brief is free of anchoring.
G2 — Requirements and evidence ledger Build traceable requirements and classify material inputs using the labels in §2; record provenance, scope, freshness where known, contradictions, and test status. Every consequential requirement and claim has an owner/source/status and a verification path, or is explicitly unresolved.
G3 — Independent option generation Generate genuinely distinct candidates for consequential choices from the requirements — not from a feature checklist. Include a simple baseline. Explain when only one option is feasible. Evaluation criteria are stated before candidate scoring or selection.
G4 — Feasibility and comparative evaluation Compare credible alternatives on the same criteria; quantify costs and constraints where data support calculation; identify trade-offs and sensitivity to unknowns. The selected design is justified against declared criteria; hard constraints are treated as gates, not compensable score items.
G5 — Design synthesis Specify behavior, boundaries, interfaces, state ownership, invariants, failure modes, dependencies, and staged implementation at the level required for engineering. No critical component is an unexplained placeholder; no critical behavior depends on an unstated assumption.
G6 — Independent adversarial review Attempt to falsify the proposed design; inspect contradictions, omitted alternatives, unsafe states, statistical flaws, unsupported claims, and unnecessary complexity. Apply D13 (Positive Derivation) to the review itself. Each material finding is resolved, contained by an explicit safe fallback, or retained as a named limitation with consequences.
G7 — Verification and revision Apply deterministic checks, calculations, source checks, or tests that are available and proportionate; correct defects and re-check affected requirements. Every critical criterion has a truthful status and evidence; changes have been checked for relevant regressions.
G8 — Final acceptance Run the audits in §§11–13, the completeness declaration in §2.2.5, and assemble a traceable decision, evidence, and acceptance record. The final status distinguishes passed, failed, blocked, and untested items; no claim exceeds the evidence; the completeness declaration is present and satisfies D20/D21/D22.

Mandatory pre-sequencing step (D11). Before executing any gate, the model MUST:

1. List the material exit conditions that must be satisfied for the design to be defensible.
2. For each exit condition, identify its prerequisites (what other exit conditions or evidence must exist first).
3. Build a directed acyclic graph (or explicitly mark cycles) of exit conditions.
4. Determine the execution order from that graph, not from the G-numbering.
5. Record the derived order and the dependency reasoning as a visible artifact.

A gate may be partially executed if some exit conditions are independently satisfiable. A gate may be reopened if a later gate's evidence contradicts an earlier conclusion. A gate may be skipped if dependency analysis shows its exit conditions are already satisfied by other work — with recorded rationale.

A gate may be blocked by an unavailable source, owner decision, or execution tool. A blocked gate is not a reason to fabricate completion or freeze unrelated work: proceed on independent tasks, mark dependent claims accordingly, and state the exact blocker and its consequence.

2.1.3 Choose actions by expected value, not by ritual

At each significant decision point, identify the most consequential remaining uncertainty or failure mode and choose the next action that is most likely to improve correctness, feasibility, risk control, or decision quality relative to its cost. Candidate actions may include targeted source inspection, calculation, a small reproducible test, counterexample search, alternative generation, implementation-level specification, or independent review. Do not browse broadly, call additional models, create more artifacts, or repeat a test merely to appear thorough.

For every high-impact decision, state the decision criteria before comparing candidates. Treat hard requirements and essential safety invariants as pass/fail constraints; do not let a high aggregate score compensate for violating them. For decisions where more than one credible option exists, compare at least two materially different options, including a simpler baseline when relevant. If only one feasible option remains, explain why rather than inventing a weak competitor.

Use calculations or executable checks when they can settle an issue more reliably than prose. Before executing a test, identify the claim it tests, the inputs/fixture, expected result, actual result, and what the test cannot establish. A test that cannot distinguish a correct implementation from a faulty one is not meaningful validation.

2.1.4 Maintain an inspectable evidence and decision record

For consequential items, maintain concise traceability across:

requirement or question → evidence/status → alternatives → decision and rationale → verification method → observed result or unresolved dependency

Record enough detail to permit a competent reviewer to reproduce the conclusion, but do not produce an exhaustive transcript of private internal reasoning. Provide decision summaries, assumptions, equations, sources, calculations, counterexamples, test results, and uncertainty statements instead. Where evidence changes a prior decision, note what changed and why; do not silently rewrite the historical rationale.

Decision reversibility classification. Every material decision in the decision record MUST be classified by its reversibility. This classification is a required field in the decision record's traceability format, and it directly feeds the pause decision in §2.1.7.

· REVERSIBLE — can be changed later without rework (e.g., naming, formatting, documentation structure, comment style). No pause needed; document and proceed.
· COSTLY_REVERSIBLE — can be changed but requires rework (e.g., component boundary between two modules, choice of a specific data schema once data is written, choice of an API version once an integration is tested). Document trade-offs; proceed if no blocker; pause if §2.1.7 conditions are met.
· IRREVERSIBLE — cannot be undone without significant loss (e.g., a decision to use a specific exchange endpoint once orders are placed, choice of a persistent state pattern once state is written, a legal/regulatory position taken). Pause and present to the owner per §2.1.7 regardless of magnitude, because reversal cost is high.

Why this matters. A reversible decision can be made quickly and corrected later. An irreversible decision that is made without owner awareness and later found to be wrong can invalidate the entire design. The reversibility class prevents both errors: it avoids pausing for trivial reversible choices, and it forces pausing for consequential irreversible choices.

Interaction with §2.1.7: the pause conditions in §2.1.7 include "material impact" and "owner-level." A IRREVERSIBLE decision that is also owner-level automatically satisfies both, regardless of whether it appears trivial at first glance.

Use explicit status values consistently: PASS, FAIL, BLOCKED, and NOT TESTED for acceptance criteria; and the evidence labels in §2 for claims and inputs. PASS requires observed evidence that satisfies the declared criterion. Absence of a known failure is not a pass. BLOCKED is appropriate when a required dependency or authorization is unavailable. NOT TESTED means the check has not been executed. Do not combine these states into a vague phrase such as "mostly validated."

Every status label MUST be accompanied by complete content per §2.2.2 (D20). A status label without content is a status-only entry and violates D20.

2.1.5 Perform an independent review where it can add information (D7)

After a candidate design exists, conduct a red-team review that tries to disprove it rather than merely improve its presentation. Inspect the strongest alternative, key assumptions, failure boundaries, hidden coupling, ambiguous outcomes, data leakage, model-selection bias, security boundaries, resource saturation, recovery paths, and every claim of readiness.

If another capable model/reviewer is actually available and its use is allowed, prefer a staged review: first ask it to derive an independent baseline from the requirements and evidence without showing it the candidate design; then disclose the candidate and ask for a structured comparison and falsification attempt. A separate reviewer is optional, not a prerequisite. Reviewers may share the same blind spots; their agreement is not proof. Resolve disagreements against requirements, source evidence, or tests, not by majority vote or rhetorical confidence.

Apply D13 (Positive Derivation) to the review itself. The review must state positive reasons for findings, not merely "the model was not told to do X."

A finding that produces a material design change MUST be propagated to every affected part of the design before the engagement closes. If the adversarial review (in the current part) finds a gap and produces a design change (e.g., "the state machine is extended to include protective-order states"), the affected part (e.g., the state-machine table in an earlier part) MUST be updated to reflect the change before the completeness declaration is signed. It is not sufficient to state in the review's finding that "the affected section is extended" — the section itself MUST contain the extension. A finding that names a design change without the change appearing in the referenced section is a phantom resolution and is a D20 violation. The completeness declaration MUST include a "material design changes from adversarial review" section that lists each finding-produces-change pair as: (a) finding ID, (b) section that was changed, (c) location within the section (line, table row, paragraph), (d) verification that the change is actually present (quote or paraphrase of the new content). If a finding's resolution cannot be propagated to the affected section because the affected section was already delivered, the design MUST NOT claim the finding is "resolved" — it MUST list the finding as "resolved pending propagation" with the explicit consequence that the affected section is currently inconsistent with the resolution.

2.1.6 Revise on evidence and stop on defensible grounds (D8)

When a material defect is found, correct it if possible and rerun affected checks. Inspect plausible regressions and confirm that previously satisfied hard constraints remain satisfied. Do not restart the whole task when a targeted correction is enough; do not make local edits that leave a cross-component contradiction.

Stop only when all mandatory requirements have an explicit status, critical acceptance criteria pass or their failure/blockage is disclosed, known critical defects are resolved or safely contained, the completeness declaration in §2.2.5 is delivered, and the remaining work has been assessed against its likely value and available budget. Continue when a cheap, high-value check remains or a critical claim is still unsupported. Stop when remaining work is redundant, infeasible, low-value, or cannot be performed with the available resources. Do not use a fixed number of iterations as a proxy for quality, and do not iterate indefinitely in pursuit of unverifiable perfection.

"I ran out of time," "I ran out of space," and "I ran out of content" are NOT valid stop conditions. If the content is long, use §2.2.4 (D22) multi-message delivery. An interruption that leaves material findings only in working memory is a completeness failure per §2.2.8, not a valid stop. Only the conditions above are valid stop conditions.

The final report must state what was actually inspected, calculated, executed, reviewed, and tested; what was not; which claims are supported; the most important remaining uncertainties; and the completeness declaration per §2.2.5. Never state or imply that following this protocol guarantees a globally optimal design, profitability, or flawless model behavior.

2.1.7 Pause-and-present protocol for pre-undeclared owner decisions

This section defines when the model MUST pause mid-engagement to present options to the owner. It distinguishes cases where pausing is required from cases where it is prohibited.

When pausing is required (MUST pause)

A material decision requires the owner's input — and the model MUST pause — when ALL FIVE of the following conditions hold:

1. Material impact. The decision materially affects the final design or the verification plan. Minor stylistic choices do NOT qualify.
2. Owner-level. The decision is one of: a business policy, a risk appetite, a legal/regulatory position, a capital commitment, an authorization boundary, or a preference between materially different trade-offs that the owner has not declared. Decisions the model is delegated to make (per §3.1 item 12, §9.1) do NOT qualify

— those are model decisions, not owner decisions.
3. Pre-undeclared. The owner has not already established the choice in §3.1, §3.2, or §3.3, and it is not derivable from existing requirements or evidence.
4. Blocking. Proceeding past the decision without resolving it would force the model to either (a) make an implicit assumption not authorized by requirements, or (b) defer material work past the natural next phase.
5. Safe pause. The pause will not lose accumulated artifacts or findings, because they have been persisted per §2.2.8.

What to present when pausing

When the model pauses, it MUST present all of the following:

· All material options the model considered, with a one-line description of each.
· Trade-offs of each option against the relevant criteria (cost, risk, complexity, feasibility, reversibility per §2.1.4).
· The model's recommendation with positive rationale (per D13) — not "this is standard," but "this option best satisfies requirement R-NNN because..."
· The exact decision requested — a single question or a small set of mutually exclusive choices.
· The default if no decision is provided — if the model can proceed safely with a PROVISIONAL flag, state it; if the model cannot proceed at all, state that.
· The consequence of delay — what work remains blocked until the decision arrives.
· The reversibility class of the decision per §2.1.4 (REVERSIBLE / COSTLY_REVERSIBLE / IRREVERSIBLE).

When pausing is prohibited (MUST continue)

The model MUST NOT pause when:

· The decision is delegatable to the model under §3.1 item 12 (architecture, method selection, feature selection, optimizer design, programming language, framework, runtime, libraries). Asking the owner to make a delegated decision violates §3.1 item 13.
· The decision is already established in §3.1, §3.2, §3.3, or prior owner declarations. Re-asking is unnecessary.
· The decision can be safely deferred — worked around with a PROVISIONAL or UNRESOLVED label — without blocking material downstream work. Continue per §1.6 and §4.2.
· The decision is trivial (stylistic choice, naming convention, formatting detail). Make the choice and label it DESIGN_JUDGMENT.
· Pausing would lose accumulated artifacts or findings. Instead: persist first (per §2.2.8), then continue with a PROVISIONAL or UNRESOLVED label, and flag the pending decision at the next natural phase transition.

Reconciliation with §1.6 and §3.1 item 13

§1.6 requires continuing work when an input is unresolved. §2.1.7 requires pausing when a decision meets the five pause conditions. These do NOT conflict because:

· §1.6 addresses declared-unresolved inputs (the owner has already said "I'll decide later"). The model continues parametrically.
· §2.1.7 addresses pre-undeclared discoveries (the model encounters a new owner-level decision mid-design that was not anticipated). The model pauses and presents.

§3.1 item 13 prohibits unnecessary questions about decisions the model is delegated to make. §2.1.7 only requires pausing for owner-level, non-delegable decisions. Asking a delegated question is still prohibited.

Example of a pause (required)

During design synthesis, the model discovers that the strategy's intraday requirement (A6) is compatible with two materially different operating modes: (a) fully automated entries and exits, or (b) automated entries with operator confirmation on exits. Both are valid designs; each has different risk, latency, and operational burden profiles. The choice is an owner preference (operator burden tolerance) that the prompt has not declared. Pause and present both options with trade-offs and the model's recommendation.

Example of a continuation (prohibited pause)

During design synthesis, the model considers whether to use a relational database or a time-series database for storing bars. §9.1 states that data storage class is a decision the model is delegated to make. Do not pause. Derive the answer from requirements and proceed.

Interaction with §2.1.6 (stop conditions)

A pause under §2.1.7 is not a stop. The engagement continues after the owner's decision. The stop conditions in §2.1.6 are about ending the engagement; §2.1.7 is about ending a message and awaiting input.

Interaction with D22 (multi-message delivery)

A pause produces a partial delivery, not a truncated delivery. The pause message MUST explicitly state: (a) what has been delivered so far, (b) what decision is requested, (c) what remains after the decision arrives, (d) the model's recommendation. This is a legitimate D22 partial delivery with an explicit deferral to owner

input — not a D21 truncation.

2.2 Completeness, anti-abbreviation, and multi-message delivery mandate

2.2.1 Purpose and scope

This section establishes three binding obligations that are independent of, and additional to, the quality gates in §2.1:

1. Completeness obligation (D20): every material requirement, deliverable, decision, and piece of evidence must be fully presented — not summarized, not abbreviated, not deferred, not truncated.
2. Anti-abbreviation obligation (D21): specific linguistic and structural patterns that reduce content are prohibited, regardless of output length, audience, or perceived time pressure.
3. Multi-message delivery obligation (D22): if the complete output exceeds a single-message limit, the model MUST deliver it across multiple sequential messages, each complete in itself, with no content deferred to a "later" message that may never come.

These obligations are hard constraints. They are not preferences. They are not overridden by output-length concerns, by the model's own judgment about what is "essential," or by the perception that the reader "gets the idea."

No form of content reduction is acceptable at any stage of this engagement. This prohibition is absolute and applies to every output, every section, every table, every calculation, every rationale, and every status record. Specifically prohibited, without exception:

· Compression (reducing length while claiming to preserve content)
· Simplification (removing nuance or complexity that the material requires)
· Summarization (presenting a shorter version in place of the full version)
· Abbreviation (shortening through omissions or substitutions)
· Omission (leaving out material items that the requirements require)
· Referral (pointing to a source or section instead of presenting the content)
· Superficiality (covering a topic at a level insufficient for engineering, review, or reproduction)
· Deferral (marking content as "to be addressed later" without an actual subsequent delivery)
· Truncation (stopping mid-content without explicit completion)

The only legitimate exception is an item explicitly marked UNRESOLVED with (1) stated reason, (2) stated resolution method, (3) stated authorization needed, (4) stated consequence — per §2.2.3's escape clause. UNRESOLVED is not a summary; it is an honest disclosure.

Any output that violates this prohibition is incomplete by construction and cannot satisfy the completeness declaration in §2.2.5.

2.2.2 The completeness obligation (D20)

What "complete" means. A material item is fully presented when:

· The item's content is present (not a reference to it, not a summary of it, not a pointer to where it could be found).
· The item's rationale is present (why this decision, why this evidence, why this exclusion).
· The item's status is present (per §2.1.4: PASS, FAIL, BLOCKED, or NOT TESTED).
· The item's evidence pointer is present (source, calculation, or test that supports the status).
· The item's consequence is present (what follows if the status changes, what depends on this item).

What "complete" does not mean. The completeness obligation does not require:

· Every possible item to be covered (only those that are material per the requirements).
· Every alternative to be exhaustively enumerated (only those that are credible per §1).
· Every edge case to be described (only those that materially affect correctness, safety, or acceptance).
· Every section to be equally long (depth follows materiality, not equal treatment).

The completeness test. Before declaring any material item "presented," the model MUST verify:

Test	Question	If NO
T-C1	Is the content directly visible in the output?	NOT complete — present it
T-C2	Is the rationale directly visible?	NOT complete — present it
T-C3	Is the status explicitly labeled?	NOT complete — label it
T-C4	Is the evidence pointer directly visible?	NOT complete — present it
T-C5	Is the consequence directly visible?	NOT complete — present it

Prohibited substitutes. The following do not satisfy D20, even if they superficially appear to:

· ✖ "See §X for details" — where §X does not contain the actual details.
· ✖ "As discussed above" — where "above" is a heading, not the actual discussion.
· ✖ "The full list is available in the source" — where the source is not accessible in the output.

· ✖ "Detailed analysis follows" — where the analysis is replaced by a summary.
· ✖ "For brevity, this is summarized as..." — the summary is not the full content.
· ✖ "In essence, ..." — the essence is not the full content.
· ✖ "Key points: [bullet list]" — the key points are not the full content.
· ✖ A table row that says "see attached" — where nothing is attached.
· ✖ A diagram without a textual explanation of what the diagram shows.
· ✖ A code block without a textual explanation of what the code does and why.
· ✖ A test result without the test specification, inputs, expected result, and actual result.

Interaction with status values. Every status must be accompanied by the complete content it labels. A status label without content is a status-only entry and does not satisfy D20. The model MUST NOT produce status-only entries.

Interaction with D18 (material-question justification). D18 requires every section to answer a material question. D20 requires the answer to be complete. Together: a section exists because it answers a material question, and when it exists, it answers fully.

Interaction with P1 (quantitative completeness pressure). D20 says "if you include an item, present it fully." P1 says "include all items." D20 governs D18 (section exists only if material). P1 is neutralized by D18 + D20 together: a material section is fully presented; a non-material item is not included at all (not included-and-summarized).

2.2.3 The anti-abbreviation obligation (D21)

Prohibited patterns. The following patterns are prohibited in the output, regardless of context, length, or perceived pressure:

Language-level prohibitions:
for brevity · for conciseness · as space permits · in summary form · in short · to summarize · in essence · essentially · broadly speaking · in a nutshell · the gist is · the short version · bottom line · key takeaway (when used to replace full content) · TL;DR (when used to replace full content) · in summary (when used to replace full content) · to recap (when used to replace full content) · as noted earlier (when "earlier" does not contain the actual content) · as will be shown (when "will be shown" never actually shows it) · will be covered later (unless "later" is an actual message sent in this engagement) · beyond the scope of this response (unless explicitly marked UNRESOLVED with resolution path) · given space constraints · to keep this manageable · abbreviated version · high-level overview (when presented as a substitute for detail) · at a high level (when presented as a substitute for detail) · simplified / simplified version (when presented as a substitute for detail) · draft (when presented as the final deliverable without an explicit plan to replace it) · placeholder (without explicit plan for replacement) · TBD (without explicit owner decision or resolution path) · to be detailed later (unless "later" is an actual subsequent message)

Structural-level prohibitions:

· Presenting a list of items when the requirement calls for explanation of each item.
· Presenting a table when the requirement calls for narrative.
· Presenting a diagram when the requirement calls for textual specification.
· Presenting a single example when the requirement calls for multiple credible alternatives.
· Presenting a sample when the requirement calls for the full set.
· Presenting a reference when the requirement calls for the content itself.
· Presenting a schedule when the requirement calls for the plan.
· Presenting a roadmap when the requirement calls for the implementation sequence.
· Presenting a checklist when the requirement calls for the completed work.
· Presenting a template when the requirement calls for the filled-in document.
· Presenting a stub when the requirement calls for the artifact.

Interruption-level prohibitions:

· Stopping mid-sentence or mid-thought without explicit continuation in the same message or next message.
· Using "..." or "etc." to end a list without explicit explanation of what is omitted.
· Ending a response mid-analysis without a clear "Part N/M" marker and a complete "Part N" delivery.
· Claiming "the design is complete" when any material requirement has status NOT TESTED or BLOCKED without explicit disclosure.

Escape clause. The only legitimate escape from D21 is an item marked UNRESOLVED with: (1) a stated reason for being unresolved, (2) a stated method for resolving it, (3) a stated authorization needed, (4) a stated consequence of remaining unresolved. UNRESOLVED is not a summary. UNRESOLVED is an honest statement that the item cannot be resolved with current resources. The distinction:

· ✖ "Summary of X: [brief summary]" — this is a summary, prohibited.
· ✔ "X: UNRESOLVED. Reason: source Y not accessible. Method: access Y via Z. Authorization: owner decision on W. Consequence: decision D remains provisional." — this is honest reporting.

Interaction with "output-length concerns." The model MUST NOT interpret output length as a reason to abbreviate. If the complete content would be long, the model MUST use D22 (multi-message delivery). Length is a delivery logistics problem, not a content completeness problem.

Interaction with "reader's time." The model MUST NOT interpret the reader's time as a reason to abbreviate. The reader has explicitly requested completeness. Abbreviating on his behalf is a violation of his request.

Interaction with "perceived importance." The model MUST NOT interpret its own judgment about what is "important" as a reason to abbreviate. Materiality is determined by the requirements, not by the model's perception.

Interaction with "confidence." The model MUST NOT interpret its own confidence (high or low) as a reason to abbreviate. High confidence does not justify omitting the rationale. Low confidence does not justify omitting the uncertainty statement. Both must be present.

2.2.4 The multi-message delivery obligation (D22)

When D22 activates. D22 activates when the complete content (per D20) cannot fit in a single message.

How D22 works. The model MUST:

1. Plan the message count before sending the first message. The plan must be visible in the first message: "This design will be delivered in N parts: Part 1 covers X, Part 2 covers Y, ..., Part N covers Z."
2. Fit-check evidence is mandatory. The initial message-count plan MUST state the evidence for the count: either an explicit estimated token/character count for the total deliverable against the known message limit, or a declared length-tracking method used during drafting. "The complete deliverable fits in one message" without evidence is a PROVISIONAL claim, not a PASS claim, and must be labeled accordingly.
3. Deliver each part completely. Each part must satisfy D20 for its scope. No part may end with "continued in next part" for content that has not yet been written.
4. Number each part explicitly. Format: Part 1/N, Part 2/N, ..., Part N/N. The numbering must be visible at the top of each message.
5. Include a completeness declaration in each part. Format: "Part K/N is complete for its declared scope. Content deferred: [none | explicit list of deferred items with reason]." If nothing is deferred, the declaration must say so explicitly.
6. Cross-reference between parts. Each part must indicate what came before and what comes after: "This part continues from Part K-1/N, which covered [X]. This part covers [Y]. Part K+1/N will cover [Z]."
7. Deliver the final part with an overall completeness declaration. Format: "All N parts have been delivered. The complete design is now present. Overall completeness status: [see §2.2.5]."

Prohibited D22 patterns:

· ✖ "Part 1/N" followed by a summary, not full content.
· ✖ "Part 1/N" followed by "the rest will be in Part 2/N" for content that Part 1/N could have included.
· ✖ Skipping parts (going from Part 1/N to Part 3/N).
· ✖ Reordering parts without explicit statement.
· ✖ Ending at Part K/N where K < N without explicit "the remaining parts are withheld due to [reason]" — which would be a BLOCKED status, not a delivery.
· ✖ Claiming "complete" in Part N/N when any material item is NOT TESTED or BLOCKED without explicit disclosure.

Interaction with the "no truncation" rule. D22 makes truncation structurally impossible. Truncation means stopping mid-content. D22 means the content is completed across parts. The two are incompatible: if the model is truncating, it is not doing D22; if the model is doing D22, it is not truncating.

Interaction with multi-turn structure. Each part is a separate message in the conversation. The model MUST NOT combine multiple parts into one message "for efficiency." The model MUST NOT split a single part across messages "for readability." The part boundary is determined by content completeness, not by message-size optimization.

Interaction with §2.2.8 (incremental persistence). D22 requires complete delivery across multiple messages. §2.2.8 requires that delivery be incremental within each phase, not accumulated. Together: each phase's artifacts are delivered as standalone messages before the next phase begins.

2.2.5 The completeness declaration

Mandatory at engagement end. Before declaring the engagement complete (per §2.1.6), the model MUST produce a completeness declaration with the following structure:

```
COMPLETENESS DECLARATION
========================
Engagement scope: [brief scope statement]
Total parts delivered: N
Total material requirements in register: R
Material requirements with status PASS: A
Material requirements with status FAIL: B
Material requirements with status BLOCKED: C
Material requirements with status NOT TESTED: D
Material requirements with status UNRESOLVED: U
Material items intentionally excluded with rationale: E
```

```
Material items not relevant with rationale: F
(A + B + C + D + U + E + F must equal R)
D20 compliance: [PASS if all status labels are accompanied by complete content]
D21 compliance: [PASS if no prohibited pattern appears in output]
D22 compliance: [PASS if all parts were delivered completely]
Artifact registry status: [PASS if all artifacts are listed and delivered per §2.2.8]
Known limitations: [list]
Next highest-value validation action: [specific action]
```

The completeness declaration is itself a material item. It must satisfy D20 (complete content, not summary). It must not be abbreviated. It must not be deferred.

All arithmetic in the completeness declaration MUST be computable and verified. Every sub-total, every sum, every cross-check, and every "adjusted for X" correction MUST be shown as a fully-expanded computation whose result the reviewer can independently reproduce. A declaration that states A + B + C + D = R MUST either produce a sum that literally equals R, or MUST explicitly show the correction (e.g., "A + B = 52; 12 items are dual-labeled and appear in both A and B; unique count = 52 − 12 = 40; discrepancy vs R = 6; see §13.6 for the disclosure of the 6 missing items"). A declaration that claims equality without the arithmetic resolving to equality is a D20 violation of the same severity as a status-only entry. This same rule applies to every calculation in the design: an arithmetic result that contradicts its own stated inputs is a hard defect, not a rounding issue. Examples: 500 bp × 14,400 s = 720 s (not 72 s); 0.05 × 3,600 = 180 s (not 18 s); a percentage-point figure that does not resolve to the raw quantity it claims to represent must be flagged and corrected. The self-audit at §13.3 MUST include an arithmetic-reproducibility walk over at least three substantive calculations in the design, and any of those three whose arithmetic does not verify is a §13.3 FAIL.

2.2.6 Self-verification protocol

Before sending each part, the model MUST verify:

Check	Question	If NO
V-1	Does this part satisfy D20 for its declared scope?	Complete it before sending
V-2	Does this part contain any D21-prohibited pattern?	Remove and rewrite
V-3	Does this part's numbering match the plan?	Correct the plan or the part
V-4	Does this part's cross-reference to prior/next parts match reality?	Correct the reference
V-5	If this is the final part, does the completeness declaration exist?	Add it
V-6	If this is the final part, does the completeness declaration satisfy D20/D21/D22?	Complete it
V-7	Have all artifacts for this part been added to the artifact registry?	Update the registry

Before declaring the engagement complete, the model MUST verify:

Check	Question	If NO
V-8	Have all N parts been delivered?	Deliver the remaining
V-9	Does the completeness declaration exist and satisfy D20/D21?	Complete it
V-10	Does the completeness declaration honestly reflect the status of every material requirement?	Correct it
V-11	Are there any D21-prohibited patterns anywhere in the delivered output?	Remove and resend the affected part
V-12	Are all artifacts listed in the registry actually delivered?	Deliver the missing ones or mark them BLOCKED with reason

Individual enumeration of V-1 through V-12 is mandatory. A statement of the form "V-1 through V-12 were performed" without an explicit per-item listing is a status-only entry at the self-verification layer and violates D20. The final self-verification record MUST present V-1 through V-12 as a table with one row per check and columns: check ID, question answered, result (PASS / FAIL / N/A with stated reason), evidence pointer (which part and section satisfies the check, or which specific artifact demonstrates it), and — for any FAIL — the corrective action taken. A row with a PASS and no evidence pointer is a status-only entry and is treated as FAIL. The purpose of individual enumeration is to make the self-verification auditable: a reviewer must be able to open the record and, for each V-N, either confirm the evidence or file a defect against a specific check, not against "the self-verification" as a whole.

Self-verification is not optional. A model that skips self-verification is not doing D20/D21/D22. The self-verification is itself a material item that must satisfy D20.

2.2.7 Interaction with other sections

§2.1.1 (Capability inventory): The capability inventory must honestly state whether the model's environment allows multi-message delivery. If the environment forbids multiple messages, this is a BLOCKED condition for D22, and the model must record it explicitly.

§2.1.2 (Gates): Gate G8 (Final acceptance) requires the completeness declaration from §2.2.5 as part of its exit condition.

§2.1.4 (Status values): Status values must be accompanied by complete content per D20. A status value without content is a status-only entry, which violates D20.

§2.1.6 (Stop conditions): The model MUST NOT stop before the completeness declaration is delivered. "I ran out of time" is not a stop condition. "I ran out of content" is not a stop condition. Only the conditions in §2.1.6 are valid stop conditions.

§2.2.8 (Incremental persistence): The self-verification MUST include a check that all artifacts referenced in the completeness declaration have been delivered incrementally per §2.2.8, not accumulated for a single final delivery.

§11 (Validation): The validation matrix must satisfy D20. Each criterion's verification method, required evidence, acceptance condition, status, observed result, and evidence pointer must all be completely present. A validation matrix with rows like "PASS — see above" violates D20.

§12 (Deliverables): The deliverables list is a coverage checklist, not a content template (per D12, defined in §12). But if a deliverable is included, it must satisfy D20. The interaction is: D12 governs inclusion; D20 governs presentation. An item can be excluded per D12 with rationale; if included, it must be complete per D20.

§13 (Self-audit): The self-audit must verify D20/D21/D22 compliance. A self-audit that claims "all complete" without the completeness declaration violates D20.

2.2.8 Incremental artifact persistence and resumability

The model MUST persist artifacts incrementally throughout the engagement, not only at completion. Context windows can be truncated, token budgets can be exhausted, sessions can time out, and any number of unforeseen interruptions can terminate the engagement mid-process. An engagement that produces excellent artifacts in a single final message but loses them to an interruption has produced zero artifacts.

Required persistence protocol:

1. Incremental delivery within each phase. As each phase produces a material artifact (requirements register, option comparison, design section, verification matrix, etc.), the model MUST deliver the artifact as a standalone message or clearly-marked message segment before proceeding to the next phase. Do not accumulate all artifacts for a single final delivery when interruption is possible.
2. Artifact registry. The model MUST maintain a running artifact registry (delivered as part of each phase's output) listing: artifact ID, artifact name, phase produced, message reference (Part N/M or section number), status (DELIVERED / DEFERRED / BLOCKED), and one-line description. This registry is itself a material artifact and MUST be updated with each phase.
3. Resumability. At any interruption point, a fresh model instance reading only the delivered artifacts MUST be able to determine: (a) what has been completed, (b) what is pending, (c) what decisions remain unresolved, (d) what the next logical action is. The artifact registry, the partial completeness declaration, and the requirements register together provide this resumability.
4. No progress loss. The model MUST NOT hold any material finding, decision, calculation, or design choice exclusively in its "working memory" (context) without persisting it to delivered output. If the model has reached a conclusion but has not yet written it into a delivered artifact, the model MUST treat that conclusion as unpreserved and prioritize persisting it.
5. Checkpoint before transitions. Before transitioning between phases, the model MUST verify that the artifact registry is current and that no material finding from the completed phase is only in working memory.

Interaction with D22 (multi-message delivery): D22 requires complete delivery across multiple messages. §2.2.8 requires that delivery be incremental, not accumulated. Together: each phase delivers complete artifacts before the next phase begins, with the final completeness declaration confirming the full artifact set.

Interaction with §2.1.6 (stop conditions): An interruption that leaves material findings only in working memory is a completeness failure, not a valid stop. The stop conditions in §2.1.6 require the completeness declaration; §2.2.8 requires that the artifacts the declaration names are actually delivered.

Interaction with §4.3 (pre-research checkpoint): The pre-research checkpoint is the first artifact to persist. Subsequent artifacts follow the same pattern.

If the environment cannot persist artifacts to files: the model MUST persist them to delivered messages. Delivered messages are the fallback persistent layer. No exception is permitted on the grounds of "the environment doesn't support file persistence" — the message layer is always available when multi-message delivery is available (per §2.1.1's capability inventory).

---

3. Project brief: what the owner has and what must remain open

3.1 Binding project constraints

1. Live venue and instrument: design for the futures/perpetual-futures market on Toobit. The owner has selected Toobit because, according to the owner's established circumstances, it satisfies their simultaneous access conditions: identity verification is available to them, API access for data and bot communication is available, the keys are free, and the venue offers usable historical data and liquidity. Do not reopen the venue decision absent a material contradiction or explicit owner authorization. This venue constraint does

not limit design research to live-execution data alone. OWNER_HARD.
2. Single runtime operator interface: the Telegram bot is the sole operator interface. Runtime control, mode selection, configuration, alerts, acknowledgements, safety refusals, monitoring, and legitimate read-only review must be available through that bot. Do not introduce a parallel dashboard, mobile app, desktop client, second bot, or routine operator console as an alternative interface. OWNER_HARD.

This is an interface constraint, not a process-topology constraint, not a framework constraint, not a deployment-location constraint. The Telegram bot may be backed by any process topology (in-process handler, separate service, relay, webhook, long polling, etc.) that the requirements support. The constraint is on what the operator sees and interacts with, not on how the system is internally architected. A design with a single Telegram interface and a complex internal service topology is a legitimate candidate. A design with a single Telegram interface and a single in-process handler is also a legitimate candidate.

The Telegram interface is not a trust boundary: authenticate and authorize privileged acts; validate untrusted messages; protect against replay, injection, rate-limit, and token/API failures; and audit all privileged actions. Routine operation must not depend on Telegram being available for the system to preserve safety.
3. UTC-only computation and recordkeeping: UTC is authoritative throughout the system, including storage, logs, schedules, windows, reports, alerts, Telegram outputs, exchange communication, and audit events. Every timestamp must have a defined semantic role (venue event time, bar open/close, availability/knowability time, monotonic elapsed time, or server-anchored wall time). Use monotonic clocks for elapsed deadlines and UTC wall-clock semantics for instants. Specify behavior when clock offset or synchronization is stale, unknown, or excessive. If any local time is displayed, it must be supplementary to UTC and must not drive computation. Daily boundaries are UTC days; weekly boundaries are ISO-8601 UTC weeks unless a justified, explicit alternative is documented. OWNER_HARD.
4. No account access or live actions during this engagement: the design model must not ask the owner to share secrets or credentials, connect to their account, perform authenticated calls, alter API permissions, access funds, or submit/cancel/modify orders. The owner attests that Toobit registration/KYC, futures enablement, real trades, and API key creation/connection have previously succeeded. This attestation is not authorization for the model to access the account and does not prove all current permissions or endpoint behavior. OWNER_HARD.
5. Cross-venue data restrictions: another venue's data may be considered for research only if the owner's conditions are met: the venue does not require registration/KYC, or it permits the owner to complete all relevant activities as an Iranian user under conditions equivalent to Toobit; the data is entirely free; and the data is restricted to research/backtest and paper operation unless full, unrestricted use by the owner is established for registration/KYC, payments, withdrawals, and all other relevant activities. Legal, sanctions, contractual, and licensing questions that are not established remain owner/legal decisions; do not make legal conclusions on the owner's behalf. Gate prohibited sources away from Live data and execution paths. OWNER_HARD.
6. Intraday intent: the owner's trading style is intraday. The design must determine a defensible operational definition and maximum holding-duration policy from the actual strategy, market behavior, funding/costs, exits, risk, and validation results. Do not invent a fixed duration merely to make the specification concrete. OWNER_HARD for the intraday intent; DESIGN_JUDGMENT for the operational definition and maximum holding-duration.
7. Broad but feasible universe: the owner prefers the broadest defensible coverage of symbols and timeframes that the actual evidence, historical coverage, trial budget, rate limits, liquidity, compute/storage, and operational constraints permit. Treat the universe as two axes — symbols and timeframes — and explain which axis each binding limitation affects. Do not assume either that all timeframes are necessary or that every timeframe outside a narrow intraday set is irrelevant. Derive multi-timeframe dependencies from the selected design; do not force a particular trading concept to justify a larger universe. The universe is derived per D14 (§10), not per any anchor list in this prompt. OWNER_PREFERENCE for breadth; DESIGN_JUDGMENT for the derived universe.
8. Reference device and desired deployment priority: the reference handset is the owner's Xiaomi POCO X6, model 23122PCD1G, running Android and intended to use Termux where applicable. Evaluate feasibility separately for development/build, model training, historical research/backtesting, extended calibration, paper operation, live execution, Telegram runtime, persistent storage, and recovery.

The phone is a feasibility constraint and a compromise axis, not an architecture driver. The model MUST first derive the strategy architecture and the required compute/storage/latency envelope from requirements (§1, §9, §10, D14). It MUST then measure or bound the POCO X6's capability (per §5.2) against that envelope. It MUST then apply the compromise order below only as a compromise mechanism, not as a pre-selection.

Mandatory ordering (D15 — Envelope-First Deployment):

1. Derive the required compute/storage/latency envelope from the strategy and the operational requirements (peak load, data volume, latency budget, recovery time, availability target).
2. Measure or bound the POCO X6's capability against that envelope.
3. Match: if the device meets the envelope, deploy on-device. If not, apply the compromise order.
4. Record the match, the gap, and the compromise with explicit consequences.

Step 4b — Enumerate all feasible deployment topologies before the compromise. The model MUST list every deployment topology that the requirements permit (e.g., on-device only; on-device + one-off cloud backtest; on-device + ongoing cloud for Live; full cloud; hybrid with edge relay; any other the requirements imply), and for each, state the envelope fit (PASS / PROVISIONAL / FAIL) with the specific gap.

No topology may appear later in the design ("conditional escalation to X if Y") unless it was enumerated in Step 4b. A topology that emerges mid-design is a silent architecture driver, which violates D15. If the model finds itself introducing "we could escalate to VPS if..." — it MUST stop and add VPS to Step 4b's enumeration with full envelope fit, and state the threshold derivation.

Compromise order (applied only after Mandatory-ordering Step 3 — Match):

1. Run all necessary phases on the phone with maximum justified capability.
2. If impossible, run all except Live on the phone.
3. If still impossible, retain development/training/calibration on the phone and reduce operational

breadth first, then depth, then remove a capability.
4. If that also fails, state the unavoidable redesign and apply the same compromise order.

Compromise sub-step disclosure. A compromise sub-step — moving fewer phases than a given compromise item requires — is permitted only if the design explicitly records it as a D15 sub-step deviation. The disclosure MUST include: (a) which compromise-order item the sub-step derives from; (b) the narrower scope actually selected (e.g., "step 2 of the compromise order moves all non-Live phases to cloud; the selected sub-step moves only the initial calibration backtest — dev, paper, and live remain on-device"); (c) the specific reason the narrower sub-step suffices (typically: the only phase that failed the D15 match step is the calibration backtest; the other phases are PROVISIONAL but not FAIL); and (d) the exact escalation criterion under which the fuller compromise item would be triggered (e.g., "if a Phase-2 soak shows paper or live also FAIL on-device, escalate from sub-step T-2 to full step T-3, which is pre-enumerated in Step 4b"). An undisclosed sub-step is a D15 violation — it is a silent narrowing of the compromise order, and it defeats the purpose of the ordered compromise (which is to make the narrowing visible and reviewable, not to bury it in a topology table).

What the compromise order MUST NOT be used for:
· It MUST NOT be used to pre-select a phone-first architecture.
· It MUST NOT be used to constrain the strategy before the strategy is derived.
· It MUST NOT be used to justify rejecting a strategy that requires more compute than the phone can provide, without first checking whether that strategy is otherwise the best design.
A cloud-first or hybrid architecture that runs Live in the cloud and development on-device is a legitimate candidate. A fully-cloud architecture is a legitimate candidate if requirements justify it. A fully-on-device architecture is a legitimate candidate if the derived envelope permits it. OWNER_PREFERENCE for phone preservation; DESIGN_JUDGMENT for the derived deployment topology.
9. Telegram reliability: no stable inbound connection is assumed for the handset. Compare an arrangement appropriate to the owner's actual available infrastructure, including long polling and webhook/relay arrangements only where genuinely feasible. Account for latency, battery, continuous runtime, secret handling, network topology, and operational ownership. Telegram loss must not release a risk limit, unblock an unsafe action, or make an uncertain exchange outcome appear confirmed. Define what happens to open positions and pending orders when the operator is unreachable. OWNER_HARD for the reliability requirement; DESIGN_JUDGMENT for the specific topology.
10. Owner-proposed phases (risk-driven derivation per D16): the owner uses the vocabulary "(a) training, (b) extended initial execution/calibration, (c) paper operation, (d) Live operation" to describe intent categories, not architectural phases.
The model MUST derive the actual phase structure from the requirements, the risks, and the verification dependencies.
Mandatory phase-derivation (D16):
1. Enumerate the material risks the phase structure must address: overfit, integration failure, execution ambiguity, data-staleness, operator-error, capital-loss, regulatory, legal, cross-venue-leakage, and any risk the requirements imply.
2. Enumerate the verification dependencies (what must be verified before what).
3. Derive the minimum phase count that addresses all material risks and respects verification dependencies.
4. Name the phases. The names may match the owner's vocabulary, or may differ. Matching is not a goal.
5. Define each phase's entry conditions, exit conditions, acceptance criteria, and rollback.
The 4-phase vocabulary is a starting reference, not a template. A design with 3 phases is acceptable. A design with 6 phases is acceptable. A design with 4 phases that match the owner's vocabulary is acceptable only if the derivation supports it — not because the owner used those words.
All deviations from owner vocabulary MUST be disclosed in a single completeness table, not selectively. The table MUST include one row per owner-intended phase (training, extended calibration, paper, live — or however the owner's vocabulary is enumerated), and for each row: the owner-intended name, the design's phase name (or "merged into X" / "split into X and Y" / "removed with reason"), and the specific rationale. Selective disclosure — explaining one deviation while leaving others implicit — is a D20 completeness violation. It hides the deviations that the model did not consider worth defending.
Paper/Live selector: The owner prefers a Telegram command to select Trade → Live or Trade → Paper without an arbitrary waiting period or return hurdle. The model MUST honor this preference subject to safety interlocks (§9.9, §11). The interlocks are:
· data-validity check (§5.1, §5.5)
· reconciliation check (exchange state = internal ledger)
· risk-limit check (all hard limits defined and within bounds)
· credential and permission check (API keys valid, permissions correct)
· config-integrity check (no drift, no stale config)
· clock check (UTC offset within tolerance)
If any interlock fails, the Live transition MUST refuse with an explicit reason. The interlocks are NOT waiting periods and NOT profit hurdles. They are correctness gates. The owner's preference is honored within the correctness gates, not overridden by them. OWNER_PREFERENCE for the selector; OWNER_HARD for the safety interlocks; DESIGN_JUDGMENT for the phase derivation.
11. Capital-planning scale: use an illustrative account equity of approximately 200 USDT, with a planning range below 500 USDT, for small-capital feasibility, minimum order/notional, margin, fee, funding, slippage, concentration, and risk analysis. This is not a statement of current live equity and does not prove any strategy feasible or profitable. The illustrative scale must be used parametrically (D17): where a calculation depends on the scale, present it as a function of the scale and show sensitivity across the range. OWNER_ATTESTED for the illustrative scale.
12. System design is delegated: the owner delegates architecture, technical method selection, feature/model selection, parameter selection, and optimizer design to the model. Any adaptation, optimization,

calibration, or self-adjustment concept mentioned anywhere in owner materials — regardless of how it is described, how many concepts are mentioned, or what mechanism they propose — MUST be treated as candidate ideas to evaluate, not as design components, not as a minimum set, and not as a starting vocabulary.

The model MUST:

1. Derive from requirements whether any adaptation/optimization mechanism is needed at all.
2. If needed, derive the minimum mechanism that satisfies the requirement.
3. Evaluate the owner's proposed concepts against that minimum.
4. Include, transform, combine, or reject each concept with recorded rationale.

The count of owner-proposed concepts (two, three, or any number) is a property of the owner's brainstorming, not a design constraint. A design with zero adaptation mechanisms is acceptable if requirements justify it. A design with one is acceptable. A design with five is acceptable only if each is justified independently. No optimizer may relax an owner-approved hard risk limit.

Programming language, framework, runtime, and any library choices are among the delegated decisions. They are not owner-level decisions and MUST NOT be paused for per §2.1.7. OWNER_HARD for delegation; DESIGN_JUDGMENT for the mechanism selection.

13. No unnecessary questions: do not re-ask information explicitly answered in this prompt. Do not ask the owner to select algorithms, infrastructure, or model classes that you have been asked to evaluate. Return only truly non-delegable decisions or missing authorizations, explain why the answer cannot be inferred, and continue the rest of the design meanwhile. OWNER_HARD.

3.2 Owner preferences and proposals — not automatic design requirements

The following must be evaluated instead of copied mechanically:

· Any adaptation or optimization concept mentioned anywhere in owner materials, regardless of count, naming, or described mechanism. The model MUST derive whether any such mechanism is needed (§3.1 item 12, D13). OWNER_PREFERENCE as candidate ideas.
· A desire to maximize supported symbols and timeframes. The universe is derived per D14 (§10) with breadth as an objective and the mobile/compute envelope as a constraint; the Pareto frontier is explicit. OWNER_PREFERENCE.
· A desire to preserve the most functionality on the phone. Evaluated per D15 (envelope-first); a phone-first result is legitimate only if the envelope permits it. OWNER_PREFERENCE.
· The phase model and a simple Telegram Live/Paper selector. Evaluated per D16 (risk-driven phase derivation) and the interlock list in §3.1 item 10. OWNER_PREFERENCE.
· Potential use of cross-venue historical data, subject to the restrictions in §3.1 item 5. OWNER_PREFERENCE within OWNER_HARD restrictions.
· Potential technical indicators, price-action taxonomies, signal families, model families, dashboards/checklists, optimizer methods, or architecture patterns mentioned in any user-provided brainstorming material. OWNER_PREFERENCE as candidate ideas.

An idea is not required merely because it is familiar, sophisticated, or appears anywhere in this prompt.

3.3 Risk boundary: owner decision intentionally pending

The owner's present preferences are provisional, not an approved policy:

· Illustrative initial account equity: 200 USDT.
· Approximate maximum drawdown preference: 15–20% of initial real account equity (30–40 USDT on the illustrative 200 USDT).
· Approximate per-trade risk preference: 5% of initial equity (10 USDT on the illustrative 200 USDT).
· The owner previously illustrated 10× leverage and 100 USDT "entered" per trade from 10 USDT × 10. This mixes different quantities and must not be adopted as a position-sizing rule without explanation.

Before asking for a final decision, explain in plain but accurate terms — using worked numerical examples — the distinctions among:

· equity, initial/current equity
· notional exposure, margin, leverage
· quantity / contract multiplier, stop distance
· gross loss, fees, funding, spread/slippage
· liquidation
· per-trade risk budget
· aggregate concurrent risk
· daily loss
· peak-to-trough drawdown

For a simple linear contract, show how approximate gross P&L relates to notional exposure and price movement, and how actual loss also depends on costs, fills, contract mechanics, and liquidation. Explain that leverage does not by itself multiply dollar P&L for a fixed notional position, though it changes required margin and loss relative to margin.

At minimum, distinguish 100 USDT notional, 100 USDT margin, and a 10 USDT maximum-loss budget; show how the same notional behaves under different stop distances and why nominal stop risk is not a guaranteed realized-loss cap. Do not prescribe a trade through these examples.

Keep the risk boundary as OWNER_DECISION_PENDING. Do not silently adopt 15%, 20%, or 5% as a hard limit, and

do not silently adopt them as provisional values, sanity-check values, illustrative values in calculations, or any other implicit anchoring (D17). If a calculation requires a risk parameter, the model MUST:

1. Express the calculation parametrically (e.g., max_loss_per_trade = f(equity, risk_fraction)).
2. Present the result as a function of the parameter, not as a single number.
3. Show the parameter's effect across a range (e.g., 0.5%, 1%, 2%, 5%, 10%) to demonstrate sensitivity.
4. Never present a result as "the answer" for a specific parameter value until the owner decides.

If the model finds itself reasoning "well, at 5% risk..." — that is interference. The model MUST stop, revert to parametric form, and continue.

The same prohibition applies to trial-budget reasoning. If the model finds itself reasoning "well, at 15m probably 3-8 configs," "likely around 10-20 at 4h," "probably a few hundred at 1d" — that is interference. Trial-budget figures MUST be calculated per §5.4 (recomputation per cell, unit-consistent, per §8.4) or marked PROVISIONAL pending recomputation. A qualitative estimate ("likely," "probably," "roughly") without a shown calculation is not a derivation. It is a silent anchor.

Continue all design work not dependent on the final owner decision, and identify exactly which final calculations/tests remain contingent on it.

---

4. Source policy and research discipline

4.1 How to label evidence

For every material decision, make clear what supports it. Appropriate evidence categories include:

· direct inspection of a source, with source identity and relevant section/record;
· an owner-ratified record, including its recorded test status and scope;
· an owner attestation;
· a transparent calculation from specified inputs;
· an established general principle recalled from domain knowledge but not newly externally verified in this engagement;
· an engineering judgment;
· a hypothesis requiring testing;
· an unresolved fact requiring owner input, measurement, or a future authorized check.

Cite the exact source path/section/record when available. A filename or search hit alone is not evidence that the contents were read. Do not invent citations, DOIs, quotations, test coverage, freshness, or source identity. Use exact dates, units, and definitions when known. Avoid the label "current" unless freshness is established; otherwise say "owner-ratified snapshot" or state that the date is unknown.

4.2 Conservative handling of unknowns

For each material unknown: (a) state what is unknown; (b) explain why it affects a decision; (c) identify the method or authorization needed to resolve it; (d) choose a conservative design branch or limit scope where practicable; and (e) state the consequence if it remains unresolved. Do not invent a numerical constant simply to complete a specification. Keep unknowns out of dependent production features unless the design contains a justified safe fallback.

4.3 Independence checkpoint

Before any external research, create a concise, dated Pre-Research Design Checkpoint based only on the mandate and the already-present ratified brief. Record the actual objectives, binding requirements, open decisions, comparison criteria, and credible architectural options without choosing a final architecture.

Make the checkpoint durable before external source inspection: if file/artifact creation is available, write it to a named artifact; otherwise, present it as the first visible progress output before making external source calls. Do not merely reconstruct the checkpoint retrospectively in the final response. If the environment makes prior persistence impossible, disclose that limitation and do not claim a separately preserved pre-research record.

Do not falsely claim that this checkpoint is entirely unanchored: the owner's brief itself contains detailed domain concepts and ratified findings. The checkpoint is specifically pre-external-research, not independent of all prior context. Preserve it and amend it explicitly if new evidence changes a conclusion. Do not let the checkpoint constrain later evidence-supported changes.

Interaction with §2.2.8 (Incremental persistence): the pre-research checkpoint is the first artifact to persist incrementally per §2.2.8. It is followed by the requirements register, the option comparison, and the design sections, each delivered as a standalone artifact before the next phase begins.

---

5. Detailed ratified technical inputs

Treat the following as the owner's established inputs, but preserve the status and scope of each. Do not convert a partial, ambiguous, withdrawn, or unresolved item into a fully verified fact. Where a numeric artifact appears, apply §2.1.0 (P1 treatment): the number is a property of the owner's evidence trail, not a design target.

5.0 Reference identifier definitions

Three identifier series are used throughout this prompt. This subsection defines them all so that no C-NNN or A-NNN reference becomes dangling. These are reference labels for traceability, not requirement IDs (which are R-NNN in §6) and not evidence labels (which are defined in §2).

A-series — Owner hard constraint references (mapped to §3.1 items):

ID	§3.1 item	Constraint
A1	item 1	Toobit as live venue
A2	item 2	Telegram sole interface
A3	item 3	UTC-only computation
A4	item 4	No account access during engagement
A5	item 5	Cross-venue data restrictions
A6	item 6	Intraday trading intent
A7	item 7	Broad-but-feasible universe preference
A8	item 8	POCO X6 reference device + deployment priority
A9	item 9	Telegram reliability requirement
A10	item 10	Owner-proposed phases + Live/Paper selector
A11	item 11	Capital scale 200–500 USDT
A12	item 12	Design delegated to model
A13	item 13	No unnecessary questions

C-series — Evidence reference IDs (mapped to §5 subsections):

ID	Content	Subsection
C1	14-timeframe capability ceiling	§5.1
C2	3,500-candle cap and 1,774-bar daily sample	§5.1
C3	13-tier BTC leverage ladder	§5.1
C4	Absence of rate-limit response headers	§5.1
C5	-1007 TIMEOUT ambiguity and -1003 rate-limit code	§5.1
C6	Taker fee 0.05% vs 0.06% unresolved	§5.1
C7	Funding clamp −30 bp observed	§5.1
C8	Order-book depth non-reproducible	§5.1
C9	Liquidity snapshot: median 498,844 USDT / 134 symbols ≥ 10M USDT	§5.1
C10	Cost-per-volatility maker floor at 15m+	§5.1
C11	clientOrderId charset [A-Za-z0-9_-]	§5.1
C12	3-day outage provisional	§5.1
C13	markPrice/klines.change semantics conflicted	§5.1
C14	POCO X6 full specification	§5.2
C15	Charging ≠ App Standby exemption	§5.2
C16	Sustained runtime not measured	§5.2
C17	Cross-venue candle history to 2019-12-16	§5.3
C18	Cross-venue transfer 0.999755 at 1h, not at 5m	§5.3
C19	Storage ratio 1.25 GB per (3,500 × 14 × 10)	§5.3
C20	Deflated Sharpe unit fix: 1.5e-2 at N=100	§5.4
C21	MinTRL 19.54 yr corrected	§5.4
C22	N_max_effective = min(published, recomputed) rule	§5.4
C23	10–20 admissible configurations at 4h	§5.4
C24	Full permutation grid inadmissible ≤12h	§5.4
C25	149-record summary (134 / 10 / 3 / 2)	§5.5
C26	Historical Open Interest UNRESOLVED	§5.5

Also defined here (used elsewhere but not previously anchored):

· D1–D22 — design-rule identifiers, each defined inline in the section where it first appears (D3 §2.1.1; D7 §2.1.5; D8 §2.1.6; D11 §1; D12 §12.1; D13 §1/§9.1; D14 §10.1; D15 §3.1 item 8; D16 §3.1 item 10; D17 §2/§3.3; D18 §2.1.0; D19 §8.0; D20 §2.2.2; D21 §2.2.3; D22 §2.2.4). The letters D1, D2, D4, D5, D6, D9, D10 are not used and MUST NOT be cited as if defined.
· E1–E9, P1–P5, G0–G8, T-C1–T-C5, V-1–V-12 — each defined inline where it first appears.
· R-NNN — requirements register IDs, defined in §6.

If any identifier in this prompt is used without a definition (in this §5.0, in §2, or inline where it first appears), that is a bug in this prompt and MUST be reported before proceeding.

5.1 Toobit and trading data

Timeframe capability list — what it is and what it is not (C1 treatment):

The owner-ratified supported timeframe list is 1m, 3m, 5m, 15m, 30m, 1h, 2h, 4h, 6h, 8h, 12h, 1d, 1w, 1M. This list describes what Toobit's API can provide, not what the design should use. It is a capability

ceiling, not a design floor, not a completeness target, and not a recommendation.

The design's timeframe set is derived per D14 — Universe Derivation Protocol (§10.1). A design using only 15m, 1h, 4h is acceptable if D14's derivation supports it. A design using all 14 is acceptable only if D14's derivation supports it. The model MUST NOT treat the 14-item list as a target, as a minimum, as a default, or as a "safe choice."

Data extraction for these listed intervals is reported available using free API keys, within the stated tested scope. Treat this as the bounded set of candidate intervals. 3d is reported UNSUPPORTED_CONFIRMED on the three REST kline endpoints, with no WebSocket candle payload observed for it. 3m and 8h are reported LIVE_PASS on the tested REST/WebSocket paths. RATIFIED_FINDING.

Candle depth — cap, not guarantee (C2):

The latest 3,500 candles is a cap, not a guarantee. Actual depth depends on interval and listing. The measured 1d sample was 1,774 bars, implying approximately 4.86 years at daily cadence rather than 9.59 years. A 3,500-candle cap was measured on perpetual symbols, not spot symbols. The full 544-symbol × 14-timeframe census was not exhaustive; the recorded census covered 58 stratified symbol/endpoint/interval combinations, 36 of which reached 3,500 bars. RATIFIED_FINDING with the stated scope limitation.

Leverage ladder — tier information, not system risk limit (C3):

The ratified BTC risk/leverage ladder is reported as 13 tiers down to 1.0×, correcting an earlier "down to 5×" statement. Do not infer a maximum allowed leverage or a system risk limit from the lowest tier alone. Use only the tier information and scope actually available; otherwise mark the relevant ceiling or account/symbol-specific tier as UNRESOLVED. RATIFIED_FINDING for the ladder; UNRESOLVED for the maximum permitted.

Rate-limit headers (C4):

X-Api-Limit-* response headers were absent in the tested environment; do not base executable weight accounting on reading a response-header delta. Use conservative explicit weight accounting; any future calibration probe must be narrowly scoped, controlled, separately authorized, and never run by this design engagement. RATIFIED_FINDING.

Error codes and timeout semantics (C5):

-1007 TIMEOUT is documented and represents an ambiguous outcome; it must not be interpreted as proof that an order was not accepted. -1003 is the rate-limit breach code. Retries must be bounded with backoff, a finite retry budget, accounting, and a hard stop; never create an unbounded retry loop. See §9.7 for the order lifecycle that respects this. RATIFIED_FINDING.

Taker fee — unresolved (C6):

Taker fee is UNRESOLVED: 0.05% was documented while 0.06% was observed, and the discrepancy was not re-established. Cost feasibility should use the more conservative of those stated values until properly resolved. Do not present a single fee value as settled. UNRESOLVED with the stated conservative branch.

Funding — tail risk, not bounded cost (C7):

A funding clamp of −30.0000 bp was observed; the alleged ±24 bp cap was refuted. Treat funding as a tail cost/risk term, not as a safely bounded cost based on the refuted cap. RATIFIED_FINDING.

Order-book depth — non-reproducible (C8):

Order-book depth is NON_REPRODUCIBLE, and one-sided versus two-sided convention remains ambiguous (the record cites BTC estimates of 90.6M one-sided and 92.7M under another convention, in USDT-equivalent). Do not use such values as settled capacity facts; define the convention and carry the uncertainty. UNRESOLVED.

Liquidity snapshot (C9):

The ratified liquidity snapshot reports median 24-hour notional around 498,844 USDT and 134 symbols at or above 10M USDT. An earlier 515k/136-symbol record is superseded. Treat these as a ratified snapshot, not as timeless live liquidity. RATIFIED_FINDING with the snapshot scope.

Cost-per-volatility — corrected finding (C10):

The cost-per-volatility analysis was corrected: values at 5m / 15m are 1.42 / 0.82, not 0.97 / 0.56; the owner-ratified conclusion is a maker-only floor at 15m and above. Preserve that finding when assessing the same cost model and scope, but state assumptions and do not generalize it to unrelated symbols or conditions without support. RATIFIED_FINDING within the stated scope.

clientOrderId character set (C11):

clientOrderId supported character intersection is [A-Za-z0-9_-], narrower than an earlier proposed character set. If deterministic client order identifiers are designed, constrain them to that set and ensure

uniqueness/non-reuse within the supported scope. RATIFIED_FINDING.

Maximum safe outage — unresolved (C12):

The maximum safe outage depends on undocumented exchange retention and remains UNRESOLVED. A provisional 3-day outage limit/threshold was recorded by the owner, but the original term Class-A was not fully defined. Do not rely on an undefined class system: define any severity/change classes you use (§6) and specify what the 3-day provisional value actually governs. Do not represent it as a verified exchange retention period. UNRESOLVED with a PROVISIONAL owner value that requires a class definition.

markPrice / klines.change field semantics — conflicted (C13):

A prior record described markPrice/klines.change as a mark-price bar's own return and said index endpoints returned empty, while the later archive summary records a conflict between documented and observed semantics of this field. Treat the field semantics as CONFLICTED / UNRESOLVED unless a specific, scope-matched record clearly supersedes one account. Do not silently choose either interpretation or build a critical feature on it. If a design feature depends on this field, the design MUST either (a) mark the feature UNRESOLVED with a resolution path, or (b) implement an explicit fallback that does not depend on the disputed semantics. UNRESOLVED.

5.2 POCO X6 / Android / Termux

The owner attests that the reference handset is a Xiaomi POCO X6, identifier 23122PCD1G with the following readings from its Settings:

· SoC: Qualcomm Snapdragon 7s Gen 2, octa-core, up to 2.4 GHz
· RAM: 12.0 GB physical + 8.0 GB memory extension (not 20 GB physical)
· Android 16, build BP2A.250605.031.A3
· System version 3.0.302.0.WNRMIXM.C07
· Security update 2026-07-01
· Kernel 5.10.236-android12-9-00003-gfb24cf99ad97-ab14313284
· Baseband MPSS.DE.3.1.1-GLB-29d2055e1e
· Storage: 256 GB total, 56 GB available at the owner's check (time-sensitive snapshot)

The owner reports a healthy-performing battery, continuous charging capability, and a previous 24-hour charging test without observed problems or noticeable heating. Treat these as OWNER_ATTESTED_OPERATIONAL_OBSERVATIONS, not as an independently verified battery-health measurement or controlled stress/thermal test.

Other ratified platform inputs:

· Charging does not exempt an app from the Android restricted App Standby Bucket (C15). A proposed operational design must not assume otherwise. RATIFIED_FINDING.
· Relevant operational threats include Doze/maintenance windows, foreground-service types/timeouts, process death and exit-reason taxonomy, App Standby Buckets, OEM process management/autostart restrictions, battery-optimization exemptions, thermal throttling, storage/WAL behavior under interruption, wakelock cost, and the absence of stable inbound connectivity. RATIFIED_FINDING.
· Sustained runtime performance is not measured (C16). Current free storage, memory pressure, sustained CPU throughput, thermal throttling, background-process behavior, network stability, and recovery after process termination/reboot remain measurements or future acceptance tests. Evaluate each phase and workload separately; do not claim that a 24-hour charging test proves full workload feasibility. UNRESOLVED with the stated measurement list.

5.3 Cross-venue and storage inputs

Cross-venue candle history (C17):

Owner-ratified cross-venue public candle history reaches 2019-12-16 for 1H and 5m in the recorded scope. RATIFIED_FINDING. The specific venue from which this evidence was gathered is not named here; the model is free to consult publicly available venues for this data, subject to §3.1 item 5's conditions.

Cross-venue transfer evidence — timeframe-bounded (C18):

Measured cross-venue transfer at 1h: median basis 0.56 bp and return-transfer correlation 0.999755. The recorded conclusion is that this transfer evidence is defensible for research at 1h / 4h but not at 5m. Any use must retain source-tag lineage. Gate it away from Paper or Live wherever the owner's legal/access conditions have not been discharged (§3.1 item 5). RATIFIED_FINDING within the stated scope; OWNER_HARD for the legal/access gating.

Storage measurement example — what it is and what it is not (C19 treatment / E5 treatment):

In one measured case, 3,500 candles × 14 timeframes × 10 arbitrarily selected symbols occupied approximately 1.25 GB in a single relational database file. The 10 symbols in that measurement (BTC, ETH, SOL, BNB, XRP, ADA, DOGE, AVAX, LINK, LTC) were chosen to exercise symbol diversity in the measurement — not as a universe recommendation, not as a liquidity-ranked set, not as a market-cap-ranked set, and not as a coverage target.

Correct use of this measurement: to calibrate the storage-per-(candle × timeframe × symbol) ratio. The model MUST:

1. Derive the final universe independently per D14 (§10.1) from requirements (A1 Toobit market, C1 timeframe capability ceiling, C2 data depth, A6 intraday, A7 breadth preference, A8 mobile constraint, C23 statistical budget, C9 liquidity snapshot).
2. Compute the storage requirement for that derived universe by scaling the measurement's per-unit ratio.
3. Record the scaling formula, units, and assumptions.

Incorrect use: treating 10 symbols as a universe default, treating 1.25 GB as a target, treating the listed symbols as candidates, or treating the measurement's data shape as the design's data shape. If the design's data shape differs (e.g., adds funding, order-book snapshots, or cross-venue data), the measurement does not transfer; a new measurement or a new ratio derivation is required. RATIFIED_FINDING for the measurement; DESIGN_JUDGMENT for the scaling to the derived universe.

Jurisdiction and licensing:

Jurisdiction, sanctions, and licensing questions for cross-venue sources are not resolved by the data-transfer measurements and remain owner/legal matters (§3.1 item 5). OWNER_DECISION_PENDING.

5.4 Ratified statistical and research-methodology inputs

Deflated Sharpe Ratio — unit correction (C20):

A reported Deflated Sharpe Ratio near zero was an artifact of mixing annualized Sharpe units with a per-bar trial count. The unit-consistent ratified figure is approximately 1.5e-2 at N = 100; the original decision was unchanged, but units must remain consistent. RATIFIED_FINDING.

MinTRL — corrected and distinguished from MinBTL (C21):

The previously reported 4.99 yr MinTRL value was not reproducible from the stated PSR–MinTRL formula and appears to be a MinTRL/MinBTL conflation. The ratified corrected MinTRL is 19.54 yr; keep MinTRL and MinBTL distinct and show the arithmetic if it is used. RATIFIED_FINDING.

N_max_effective — recompute, then min (C22):

For the effective trial budget, use N_max_effective = min(published panel value, independently recomputed value) when both values exist, showing the recomputation per cell. If the two differ by more than 2×, the recomputed value governs and the discrepancy is recorded. Do not quote a trial budget from a table alone. RATIFIED_FINDING.

This prohibition applies to ALL trial-budget figures, not only to N_max_effective. Any use of 10–20 at 4h, thousands at 1d, a few hundred at X, or any other trial-budget anchor as a justification for a design decision requires either (a) a per-cell recomputation shown inline, or (b) an explicit PROVISIONAL pending recomputation label on the claim and its conclusion. Quoting an anchor for context and then using it as justification is a D21 violation, not a legitimate use.

Admissible configuration count (C23):

The prior analysis put the admissible evaluated-configuration count at 4h on the order of 10–20 even after pooling; the 1d value was in the thousands before a further sample-length correction. Pooling is limited by cross-cell dependence. The ratified conclusion is that a full permutation grid search is inadmissible at every timeframe ≤12h (C24). This constrains any exhaustive search proposal; it does not by itself select a replacement optimizer. RATIFIED_FINDING. The 10–20 and thousands figures are PROVISIONAL pending recomputation per the paragraph above until the model shows the recomputation.

CSCV arithmetic correction:

An earlier CSCV episodes-per-fold arithmetic error was corrected. The expected maximum of N independent standard normals was checked against its published anchor; the method was considered sound, but input corrections were needed. Do not reproduce the retired arithmetic. RATIFIED_FINDING.

5.5 Archive status and its limits

The owner's summary of the archived reference package describes 149 canonical records with these dispositions (C25):

· 134 LIVE_PASS
· 10 LIVE_PARTIAL
· 3 UNSUPPORTED_CONFIRMED
· 2 DOC_CONFLICT

The count sums to 149; it does not mean all 149 records passed. The recorded package status also says:

· Historical Open Interest remains UNRESOLVED (C26).
· The authenticated read-only stage was not executed.

· Offline integrity/regression gates reportedly passed.
· The full 544 × 14 census was not exhaustive.
· Three-day candles were unsupported in the tested paths.

Preserve all these limitations. If the raw archive is inaccessible to you, use this as OWNER_SUPPLIED_AUDIT_SUMMARY, not as a file you personally inspected. RATIFIED_FINDING with the stated scope limitations; UNRESOLVED for historical OI.

---

6. Requirements and unresolved decisions

Before choosing an architecture, build a compact, complete, single requirements register. At minimum distinguish:

· Objectives and success measures
· Functional capabilities and explicit non-goals
· Trading universe and timeframes (derived per D14)
· Data and lineage requirements
· Statistical validity and experiment budget
· Capital/risk controls and authorization
· Execution and reconciliation
· Telegram usability, permissions, and alerts
· Device/runtime/storage/network feasibility
· Resilience, recovery, security, and observability
· Reproducibility, maintainability, and testability
· Cross-venue legal/data restrictions
· Each assumption, owner preference, and unresolved decision

Give every material requirement a unique identifier. Each entry must contain, completely (per D20):

Field Content
ID e.g., R-001 (must be defined in this register; no dangling IDs)
Priority HARD / SHOULD / MAY
Rationale/source §section, evidence label, owner decision, or derivation
Verification method DESIGN_INSPECTION / CALCULATION / SOURCE_CHECK / DETERMINISTIC_TEST / EXPERIMENT / OWNER_DECISION
Acceptance condition explicit threshold, or UNRESOLVED + resolution method
Status PASS / FAIL / BLOCKED / NOT TESTED / UNRESOLVED
Evidence pointer source, calculation, or test result
Failure response safe fallback, rollback, or refusal behavior
Regression checks which other requirements are affected by a change to this one

Do not make up numerical acceptance thresholds. For thresholds that cannot be derived, mark them UNRESOLVED and describe the required measurement or owner decision.

The 11-field standard applies to every requirement entry, including auxiliary requirements surfaced during design (e.g., R-031 and onward in a prior run). An auxiliary requirement with only ID / Priority / Rationale / Status is a status-only entry and violates D20. Complete all 11 fields for every row, or do not enter the requirement.

The requirements register MUST be a single table. Multiple mini-registers (main + auxiliary + "see below") fragment the register and violate the D20 completeness obligation. If a requirement does not fit in the main table, add columns — do not split registers.

Undefined classes are prohibited. If the design uses severity classes, change-authority classes, outage classes, or risk classes, the register MUST define them before they are relied upon. The Class-A reference in §5.1 is an example of an undefined class and MUST be replaced with an explicit definition before use.

Utility function is not fully defined. The owner has not defined a complete utility function or guaranteed-return target. Therefore:

· Do not infer an acceptable return, acceptable probability of loss, drawdown tolerance, trading frequency, or risk appetite from the desire for a "super system."
· Compare meaningful alternatives across risk, cost, robustness, explainability, operational burden, resource use, and expected net economic value.
· If the objective depends on a non-delegable risk preference, present the trade-off and keep the final parameter pending (§3.3, D17); continue architecture work not dependent on it.
· State plainly if available data or sample size cannot support the performance claims needed to justify a proposed strategy or optimizer.

---

7. Research workflow and anti-anchoring protocol

Dependency-first ordering (D11) applies to this section. The order below is a default dependency ordering

for a typical engagement, not a mandate. Before executing any step, the model MUST perform the dependency analysis per §2.1.2. Steps may be merged, split, reordered, skipped, or run in parallel with recorded rationale. The numbering exists so the model can reference a step without re-describing it.

7.1 Default dependency ordering

Step 1 — Capability and boundary check (per §2.1.1):
Record the tools and sources actually available, the no-account-access/design-only boundary, the source perimeter, and any access limitation.

Step 2 — Pre-research checkpoint (per §4.3):
Before any external research, persist a dated checkpoint to an artifact or first visible progress output, recording objectives, hard constraints, unresolved decisions, comparison criteria, and plausible design directions. Do not select a final architecture. Do not reconstruct the checkpoint after research.

Step 3 — Targeted evidence inspection:
For each planned source inspection, name the material question it could resolve. Inspect only relevant material; preserve provenance, scope, status, and conflicts. Report what was and was not accessed.

Step 4 — Requirements and feasibility (per §6):
Map material requirements to a unique identifier, priority, rationale/source, verification method, and acceptance condition where possible. Determine feasibility under the actual historical data, statistical budget, capital scale, exchange constraints, and phone/runtime envelope.

Step 5 — Independent option generation (per §2.1.3):
Generate credible design candidates from requirements and constraints, not by copying candidate features. Include a simple baseline for consequential choices where relevant. State decision criteria before scoring candidates.

Step 6 — Comparative evaluation (per §2.1.3):
Evaluate candidates against the same criteria, identify hard pass/fail constraints, and use explicit calculations where justified. Keep forecasts, backtest findings, out-of-sample evidence, Paper behavior, and future Live evidence separate.

Step 7 — Design synthesis (per §9):
Specify behavior, data lineage, state ownership, interfaces, failure boundaries, risk authority, reconciliation, and resource use. Add only components whose purpose is justified (D18).

Step 8 — Safety and operational analysis (per §9.8):
Analyze lost Telegram connectivity, network loss, process death, stale data, clock errors, timeouts, rate limits, ambiguous exchange outcomes, partial fills, inconsistent positions, corrupt storage, unauthorized changes, and resource exhaustion.

Step 9 — Verification plan and execution (per §11):
For every critical requirement, define a meaningful test or other verification method and an acceptance criterion. Execute the checks that are actually feasible and authorized; clearly distinguish a proposed test from an executed one.

Step 10 — Independent adversarial review (per §2.1.5):
Try to falsify key assumptions and decisions, including a blind independent review where a suitable reviewer is actually available. Record, triage, and resolve findings by evidence rather than consensus. Apply D13 (Positive Derivation) to the review itself.

Step 11 — Final acceptance (per §2.1.6 and §2.2.5):
Run the audits in §§11–13, produce the completeness declaration per §2.2.5, and deliver the design, evidence/status record, unresolved-owner decisions, known limitations, and next highest-value validation step. Do not hold independent work hostage to one pending risk-boundary decision. Do not declare completion before the completeness declaration is delivered.

7.2 Adaptive sequencing

At every stage, choose the next action according to the expected value of resolving a consequential uncertainty relative to its cost (§2.1.3). Do not repeat already-ratified tests absent a material, authorized reason; do not let a step label stand in for evidence that its exit criterion has passed.

Deviation protocol. If the dependency analysis yields an order different from §7.1, the model MUST:

1. Record the deviation.
2. State the dependency that justifies it.
3. Confirm that no downstream requirement is left unverified by the deviation.
4. Include the deviation in the final evidence record.

Parallelization. Steps with no dependency between them MAY be executed in parallel or interleaved, provided each step's exit condition is satisfied before its dependents proceed.

Stop-and-present. During any step, if the model encounters a pre-undeclared owner-level decision that meets

the five conditions of §2.1.7, it MUST pause and present per that section. A pause is not a stop; it is a legitimate interruption with explicit resumption on the owner's decision.

7.3 Anti-anchoring protections (summary)

The following protections are active throughout §7:

· Pre-research checkpoint (§4.3) — prevents retrospective construction of independence.
· Dependency-first ordering (D11) — prevents the numbered sequence from becoming an execution mandate.
· Positive derivation (D13) — prevents negative anchors from being treated as exclusion mandates.
· Material-question justification (D18) — prevents section proliferation from quantitative pressure (P1).
· Parametric risk presentation (D17) — prevents silent adoption of pending risk preferences (§3.3).
· Completeness obligation (D20) — prevents abbreviation, summarization, truncation, and deferral (§2.2).
· Anti-abbreviation (D21) — prohibits language- and structure-level reduction (§2.2.3).
· Multi-message delivery (D22) — provides a complete-delivery mechanism when content exceeds a single message (§2.2.4).
· Incremental persistence (§2.2.8) — prevents loss of material artifacts to interruption.
· Evidence-first mandate (§2.1.0c) — prevents plausible-sounding content without evidence.
· Pause-and-present (§2.1.7) — ensures pre-undeclared owner-level decisions are surfaced, not silently assumed.

---

8. Quantitative and statistical integrity

8.0 Framing: rigor as verification discipline, not design driver (P5 treatment — D19)

Statistical rigor is a verification discipline applied after a strategy has been proposed. It is not a design driver that predetermines what strategies are admissible.

The model MUST:

1. Propose a strategy architecture that satisfies the requirements (§3.1 items 1, 6, 7, 11; §10 D14; and the general requirements register in §6).
2. Subject it to the statistical validity checks in §8.1–§8.6 below.
3. If it fails, either (a) revise the strategy to address the failure, (b) reduce the trial budget and re-test, or (c) report the strategy as UNRESOLVED with explicit trade-offs and consequences.
4. Never preemptively reject all strategies on statistical grounds without proposing at least one candidate and documenting why it failed.

Preemptive acceptance is symmetric to preemptive rejection and is equally prohibited. The model MUST NOT claim that a strategy "would achieve" a given statistical result, "is achievable" above a threshold, or "likely admissible" within the trial budget, before the corresponding calculation is executed and shown. Predictive claims about unexecuted statistical results are hypotheses (PROVISIONAL), not conclusions (PASS). A strategy proposed under D19 must be labeled NOT TESTED until an actual backtest or calculation is shown; any sentence of the form "X is achievable" or "X is likely" in the statistical section MUST be rewritten as "X would be the target if [calculation]; this remains a hypothesis."

Statistical rigor constrains how a strategy is validated. It does not constrain whether any strategy can be proposed. A design that begins with "the trial budget is too small for any strategy" is a failure of D19, not a demonstration of rigor. A design that begins with "here is a candidate strategy; here is why it is statistically defensible within the budget; here is what remains uncertain" is the correct pattern.

Interaction with P5 (statistical-rigor-as-axis pressure): the ratified findings in §5.4 (C20–C24) are constraints on evaluation, not preemptive exclusions. They shape how a strategy is tested; they do not shape whether a strategy is allowed to exist. See §8.4 for the admissible trial-budget handling.

8.1 Choose methods for the actual research question

Choose methods for the actual research question and data shape rather than mechanically applying a checklist. Address, as relevant to the specific design:

· point-in-time correctness, availability time, and look-ahead leakage
· survivorship/selection bias, missing and corrupted observations, listing history, and contract changes
· serial dependence, dependence across symbols/timeframes, nonstationarity, and regime dependence
· train/validation/test contamination, repeated strategy search, data snooping, multiple testing, and optimizer overfit
· realistic fees, spread, slippage, funding, latency, partial fills, order constraints, market impact, and liquidation
· parameter uncertainty, sensitivity, robustness, and uncertainty in risk/performance measures
· chronological testing, walk-forward or nested validation, purging/embargo, resampling, stress scenarios, and other techniques where their assumptions fit

The word "as relevant" is binding. If a technique is not materially relevant to the specific design, the model MUST record why it is not applied, and this record satisfies the checklist item (per D20: the item is present with rationale, not omitted silently).

8.2 Baselines, ablations, and negative controls

Use credible baselines, ablations, and negative controls where useful to estimate incremental value. Specifically:

· Baseline: a simple, defensible reference strategy (e.g., trend-following on a single timeframe; buy-and-hold; random entry with the same risk controls). The baseline's existence is not a preference for simplicity — it is a measuring stick for whether complexity earns its place.
· Ablation: remove one component at a time to estimate that component's incremental contribution. A component whose removal does not materially degrade performance is a candidate for removal (D18 — the component must answer a material question).
· Negative control: a test that should produce a null result if the design is correct (e.g., shuffled labels, random parameters, future-peeking guard test). A negative control that produces a positive result indicates a leakage or methodology defect.

Do not equate predictive accuracy with profitability. A model with high accuracy but poor risk-adjusted net return is not a good trading model. A model with modest accuracy but strong risk-adjusted net return can be excellent. Report accuracy, precision, recall, and F1 as auxiliary metrics, never as primary.

8.3 Net-of-cost reporting and risk measures

Report net-of-cost results and risk measures appropriate to the intended use. Specifically:

· Separate gross return, net return, risk-adjusted results, drawdown, turnover, tail exposure, capacity, execution sensitivity, and uncertainty. Each must appear as its own line, not as a blended composite.
· Cost model transparency: each cost term (fees, spread, slippage, funding) must be stated with its value, source, and scope. A cost that is UNRESOLVED (e.g., taker fee 0.05% vs 0.06% in §5.1, C6) must be reported as a range, not as a point estimate.
· Sensitivity: show net return under at least two cost scenarios (e.g., documented fee vs conservative fee; observed slippage vs doubled slippage). A strategy that is profitable only under the optimistic cost scenario is a candidate for exclusion, but the exclusion must be recorded with the specific scenario, not as a blanket rejection.
· Uncertainty: report confidence intervals, standard errors, or bootstrap distributions where feasible. A point estimate without uncertainty is a claim the design cannot support.

8.4 The admissible trial budget is a binding quantitative constraint

The trial budget from §5.4 is a hard constraint, not a soft guideline. Specifically:

· N_max_effective = min(published panel value, independently recomputed value) when both exist (C22).
· If the two differ by more than 2×, the recomputed value governs and the discrepancy is recorded.
· A full permutation grid search is inadmissible at every timeframe ≤12h (C24). This is a binding constraint, not a preference.
· The admissible evaluated-configuration count at 4h is on the order of 10–20 even after pooling (C23); at 1d it was in the thousands before a further sample-length correction (C23). These figures are PROVISIONAL pending recomputation per §5.4 until the model shows the recomputation.

The model MUST NOT propose an exhaustive parameter permutation search on timeframes ≤12h. It MAY propose alternatives such as:

· a small pre-registered hypothesis set (e.g., 5–15 configurations, registered before evaluation)
· sequential testing with alpha spending
· analytically constrained parameterization (parameters derived from first principles, not searched)
· staged screening with a held-out confirmation set
· Bayesian optimization with an explicit acquisition function and a bounded evaluation count
· or any other approach whose trial-budget control is explicit

None of these is automatically preferred. Each must be evaluated on its merits against the requirements, and the choice recorded with criteria declared before the comparison (§2.1.3).

Selection procedure requirements. Any selection procedure MUST declare:

Field	Content
Searched configurations	the full set of configurations evaluated, listed explicitly or by a generator rule
Effective sample size	the number of independent observations available for each configuration
Assumptions	distributional, stationarity, dependence, cost, and execution assumptions
Stopping rule	the exact condition under which search terminates
Selection-bias control	how the winner is selected (e.g., Deflated Sharpe, CSCV, walk-forward holdout)
Promotion policy	the criteria a candidate must meet to move from research to paper
Rollback policy	the criteria under which a promoted configuration is retired

A selection procedure that omits any of these fields is incomplete (D20) and MUST be marked as such.

8.5 Forecasting, calibration, portfolio, and risk are different concerns

The forecast or optimizer must never be able to relax hard risk controls. Specifically:

· Forecasting: produces a signal or probability. It does not produce a trade.
· Calibration: adjusts forecast parameters to reduce bias. It operates on forecast inputs only.
· Portfolio construction: translates forecasts into positions given constraints. It operates on portfolio-level quantities.
· Risk limits: an independent authority that can veto any portfolio or forecast. It is not a parameter that any of the above can tune.

Each of these must have its own parameter register (§8.6), its own evaluation budget (per §8.4), and its own failure mode. A design that blurs these into a single optimization is incomplete and MUST be recorded as such.

Hard owner-defined risk limits are not hyperparameters. Do not optimize a hard owner-defined risk limit as though it were a free parameter. The risk limits in §3.3 remain OWNER_DECISION_PENDING (D17); the model MUST NOT calibrate them.

8.6 Parameter register for dynamic parameters

For each material dynamic parameter, define:

Field	Content
Name	identifier used in code and documentation
Units	physical units (currency, %, ratio, count, seconds, etc.)
Meaning	what the parameter controls in the strategy
Source	owner decision, derivation, calibration, or hypothesis
Allowed domain	minimum, maximum, and constraints (e.g., must be positive)
Estimation method	how the parameter is estimated (or "fixed by owner")
Search budget	number of evaluations permitted under §8.4
Objective	what the parameter is optimized against (must not be a hard risk limit)
Constraints	what the parameter must satisfy (e.g., stop-distance ≥ minimum tick)
Calibration data	which data is used for estimation
Validation	which data is held out for validation
Uncertainty	confidence interval or bootstrap distribution
Applicability	which symbols, timeframes, regimes the parameter applies to
Promotion	criteria for moving from calibration to production
Rollback	criteria for retiring the parameter
Recalibration triggers	events (regime change, drawdown, drift) that trigger re-estimation

Or explicitly state why a field is not yet defensible. A parameter with UNRESOLVED fields is admissible only if the design contains a safe fallback that does not depend on the unresolved field. A parameter that is critical to correctness and has UNRESOLVED fields is a BLOCKED condition that must be disclosed.

Interaction with §2.1.0 (P1 treatment): the number of parameter fields is not a completeness target. A parameter with 10 well-justified fields is preferable to a parameter with 15 fields where 5 are placeholders.

---

9. Architecture discovery and technical design

9.1 No architectural dimension is predetermined (E3.1 treatment — D13)

No architectural dimension is predetermined. The model MUST derive every consequential architectural choice from the requirements, the evidence, and the design constraints.

The space of architectural dimensions is open-ended and includes, without being limited to:

· process topology (monolith, modular monolith, services, serverless, edge, hybrid)
· communication pattern (synchronous, asynchronous, event-driven, request-response, streaming)
· data storage class (relational, document, time-series, columnar, file-based, hybrid)
· deployment location (on-device, cloud, hybrid, edge, peer)
· implementation language and runtime
· model class and optimizer family
· component count and boundary definition
· repository and artifact organization
· interface style (CLI, HTTP, message queue, RPC, in-process, etc.)
· persistence pattern (event-sourced, state-machine, ledger, log-structured, etc.)
· scheduling pattern (cron, event-triggered, polled, streamed, hybrid)
· and any other dimension the requirements imply

The model MUST:

1. Enumerate the architectural dimensions that the requirements actually touch.
2. For each dimension, derive candidates from first principles. Not from a template, not from an industry default, not from "what a trading system usually looks like."
3. Evaluate candidates against criteria declared before evaluation (§2.1.3).

4. Select and justify, with explicit trade-offs, dependencies, failure modes, and what evidence could change the decision.

The model MUST NOT treat any dimension not named above as out of scope. Equally, the model MUST NOT treat any dimension named above as in scope by default. The list is a reference aid for the reviewer, not a checklist for the model.

Positive derivation (D13). Every architectural choice must be positively derived from requirements. "The prompt did not say to do X" is not a reason to avoid X. "The prompt said not to do Y" is not a reason to avoid Y if requirements justify Y. The model MUST articulate the positive reason for every consequential architectural choice and every consequential exclusion.

Prohibited reasoning patterns (per D13):

· ✖ "The prompt did not mention X, so X is out of scope."
· ✖ "The prompt listed Y as a non-example, so Y must be excluded."
· ✖ "The prompt emphasized Z, so Z must be the primary driver."

Required reasoning pattern:

· ✔ "Requirement R-NNN implies that choice C is or is not defensible because [specific rationale grounded in R-NNN or in evidence E-NNN]."

9.2 Component boundary derivation

Define component boundaries only when they create justified separation of:

· responsibility
· state ownership
· trust boundary
· deployment unit
· failure domain
· testing seam
· change authority

For each necessary component, specify (completely, per D20):

Field	Content
Purpose	one-sentence statement of why the component exists
Responsibilities	what it does
Non-responsibilities	what it explicitly does not do
State ownership	which persistent and in-memory state it owns
Inputs	what it receives and from whom
Outputs	what it produces and to whom
Dependencies	what it requires from other components or external systems
Timing assumptions	latency, ordering, and concurrency guarantees it relies on
Failure behavior	what happens when it fails, and what happens to its dependents
Verification	how its correctness is tested
Interface contract	the exact shape of its inputs and outputs

Logical responsibility does not automatically imply a separate process, package, service, engine, repository, or file. A design with 5 logical components in a single process is a legitimate candidate. A design with 5 logical components in 5 services is a legitimate candidate. The choice between them must be derived from requirements (§9.1), not assumed.

9.3 End-to-end flow specification

Specify the end-to-end flow sufficiently to cover:

· data ingestion and validation
· research inputs
· decisions
· risk approval/veto
· order intent
· execution
· exchange acknowledgement/fill
· state reconciliation
· persistent records
· monitoring
· adaptation
· rollback

Only to the extent relevant to the selected architecture. A flow that does not appear in the selected architecture does not need to be specified. But a flow that is in the selected architecture must be specified completely — no "and then magic happens" gaps.

Identify authoritative state and prevent silent divergence between the internal ledger and exchange state. For each state quantity (position, order status, balance, PnL), specify:

· which component owns the authoritative value
· which components hold a cache or replica
· how the cache/replica is validated against the authoritative value
· what happens when validation fails
· how reconciliation is triggered and completed

9.4 Data design (E3.2 treatment)

No data class is presumed necessary, and no data class is presumed unnecessary. For each data class the strategy and constraints might plausibly require, the model MUST evaluate:

1. Material incremental value — does this data class change a material decision, and how?
2. Availability — is it obtainable within the permitted source perimeter (§4) and licensing conditions (§3.1 item 5)?
3. Cost — storage, compute, bandwidth, and operational burden.
4. Correctness — point-in-time availability, look-ahead risk, quality controls.
5. Consequence of absence — what breaks if this data is unavailable, stale, or corrupted?

The model MUST justify both inclusion and exclusion for every plausibly relevant data class. A data class whose absence materially degrades the design is a strong candidate for inclusion. A data class whose absence is transparently handled is a candidate for exclusion. Both decisions must be recorded per D20.

The model MUST NOT use "common in trading" as a reason for exclusion or inclusion. The only admissible reasons are the five criteria above.

For each included data input, identify (completely):

Field	Content
Purpose	why the design needs this data
Incremental value	what decision changes if this data is absent
Availability	source, licensing, and access conditions
Timeframe and symbol coverage	which of the design's universe it covers
Timestamp semantics	venue event time, availability time, or bar close
Latency and staleness	maximum acceptable delay before the data is considered stale
Historical depth	how far back it goes
Point-in-time status	is the data reconstructed or as-published?
Quality controls	validation rules, outlier handling, gap handling
Lineage	how the data can be traced from source to decision
Retention	how long it is kept, and why
Failure behavior	what happens if the data is missing, stale, or corrupt
Storage and compute cost	per-unit and total
Consequence of absence	what breaks if the data is unavailable

Specifically for the data classes implicated by C13 (markPrice) and C26 (historical OI): if either is included, the design MUST explicitly address C13's conflicted semantics and C26's UNRESOLVED historical availability. A feature that depends on either without a fallback is UNRESOLVED and MUST be labeled as such.

9.5 Handling of missing, stale, duplicate, malformed, contradictory, unavailable data

Handle each explicitly. Never substitute a numerical zero for missing information without a justified semantic rule. Specifically:

· Missing: what is the rule for treating a gap? Interpolation, forward-fill, exclusion, or a MISSING marker? Each has consequences; state them.
· Stale: what is the maximum acceptable age before data is considered stale? What happens when stale data is encountered?
· Duplicate: how are duplicates detected and resolved? Last-write-wins, first-write-wins, or exclusion?
· Malformed: what is the validation rule? What happens when validation fails?
· Contradictory: when two sources disagree, which is authoritative? How is the disagreement recorded?
· Unavailable: what is the safe fallback when a required data source is down?

Preserve source tags for cross-venue data. Do not let a research-only source leak into Paper or Live paths where owner/legal conditions have not been satisfied (§3.1 item 5). A design that mixes source-tagged data without gating is incomplete.

9.6 Risk, portfolio, and execution

Risk controls are independent authority. Identify which conditions are:

· hard vetoes — the risk authority blocks the action, period.
· advisory alerts — the risk authority warns but does not block.
· owner-authorized changes — the owner can override, with audit.

Define safe behavior when risk inputs are missing, stale, inconsistent, or unverifiable. Unknown risk must not silently pass. A risk check that cannot be evaluated is a hard veto, not an advisory alert.

A missing, unreadable, or corrupted persistent risk-state record is an unevaluable condition, not an initialization trigger. This applies to every risk-state quantity the design persists: drawdown-from-peak, daily realized loss, aggregate exposure ledger, position-count ledger, and any other state whose absence makes a risk check unevaluable. The system MUST NOT initialize a missing risk-state record from current market or account values. Initializing drawdown_from_peak := 0 on a missing ledger is functionally equivalent to resetting a real drawdown to zero — it is a fail-OPEN path disguised as a startup convenience. The correct behavior is: (a) enter hard veto, (b) refuse Live mode because the risk-limit evaluation (§3.1 item 10, third interlock) cannot complete when the state it evaluates against is unknown, (c) alert at CRIT class, (d) require either an out-of-band restore from backup or an explicit operator acknowledgement that is separately audited and states the operator is accepting the risk of operating without ledger history. The acknowledgement is a resume gate, not a ledger substitute — the ledger remains in an unknown state and the design MUST NOT pretend it has a valid value.

Veto clearance and operator acknowledgement are two distinct operations. A veto is cleared only by its own recovery condition — the underlying state that caused the veto must actually change (the drawdown threshold, reconciliation status, data freshness, or whatever the veto class is about must return to a passing state). An operator command cannot clear a veto; it cannot make the underlying condition pass. An acknowledgement (/ack <reason> in a downstream command spec) is a resume gate that operates after the veto has been cleared by recovery — its purpose is to prevent automatic resumption on a transient recovery that might reverse (e.g., a one-bar bounce after a drawdown). The correct sequence is: veto raised → underlying condition recovers → veto clears automatically → entries remain paused because the resume-gate has not been satisfied → operator /acks → entries permitted. An /ack that arrives while the underlying condition is still failing MUST be refused with the current state, not accepted as a silent override. The design MUST specify, for each veto class, both the recovery condition (which clears the veto automatically) and whether the resume-gate applies (i.e., whether an /ack is required after recovery, or whether recovery alone resumes entries). Conflating veto and acknowledgement — treating /ack as a way to clear a veto — is a fail-OPEN design defect.

Protect hard limits from:

· optimizer (must not be able to tune a hard limit)
· signal model (must not be able to override a hard limit)
· configuration (must not be able to disable a hard limit silently)
· retry logic (must not be able to bypass a hard limit by retrying)
· operator-interface failures (must not be able to unblock an unsafe action)

Define exposure, notional, margin, leverage, contract quantity, stop-risk estimate, aggregate concurrent exposure, costs, drawdown, and other required quantities with units. A quantity without units is not a quantity. Distinguish:

· planned stop loss — the intent
· nominal stop risk — the assumed loss if the stop is hit
· realized loss — the actual loss after fills, slippage, fees, and funding
· guaranteed realized loss cap — a concept that exists only if the position is fully hedged with guaranteed fills; otherwise it does not exist

State the assumptions behind cost and fill simulation, and refuse unsupported precision. "Approximate PnL" is acceptable when the approximation is stated; "exact PnL" is a claim the design cannot usually support.

All monetary quantities are denominated in USDT (or the specific quote currency of the instrument). The design MUST NOT present a quantity as "USD" — the trading venue and the strategy universe operate in USDT-quoted perpetuals, and any USD figure is a derived approximation that must be labeled as such.

9.7 Order lifecycle and state transitions

For execution, specify completely:

· order lifecycle and state transitions — the full set of states an order can be in (e.g., INTENT, SUBMITTED, ACKNOWLEDGED, PARTIALLY_FILLED, FILLED, CANCEL_PENDING, CANCELLED, REJECTED, TIMEOUT_AMBIGUOUS, RECONCILING) and the transitions between them
· input validation — what must be true before an order intent becomes an order submission
· idempotency — how the same order intent cannot create duplicate orders
· unique client IDs — the format, generation rule, and uniqueness scope (per C11: [A-Za-z0-9_-])
· rate-limit handling — how rate limits are respected without unbounded retry (per C4: no reliance on response headers)
· retry budget — the finite number of retries, the backoff schedule, and the hard stop
· timeouts — what timeout triggers, and what it means
· partial fills — how the position is tracked and reconciled
· cancellations/races — what happens when a cancel races with a fill
· rejected requests — how rejections are recorded and whether they block further attempts
· ambiguous outcomes — the handling of -1007 TIMEOUT (C5) and any other ambiguous outcome
· protective orders — stop-loss, take-profit, and their lifecycle
· recovery — how the system recovers from a crash mid-order
· reconciliation — how the internal ledger is reconciled against exchange state

-1007 TIMEOUT (C5) means the external outcome may be ambiguous. Do not automatically resubmit as though no order existed. A timeout never proves rejection. Define the source of truth and the actions prohibited until reconciliation completes.

A complete order state machine is a material deliverable. An incomplete state machine is a NOT TESTED or BLOCKED condition that must be disclosed.

9.8 State, audit, security, and operations

Record enough to explain and, to the extent technically possible, reproduce each material decision:

· data versions and quality outcomes
· model and configuration versions
· forecast/decision
· risk approval/veto
· order intent
· exchange responses
· fills and positions
· configuration changes
· privileged operator actions
· failures and recoveries

Define required durability, immutability, retention, access control, clock semantics, and correction policy. Do not assume event sourcing or any other storage pattern without justification (see §9.1).

Security and operations:

· Secrets: how are secrets stored, rotated, and accessed? Never log secrets. Never transmit secrets over unencrypted channels.
· Least privilege: API keys must be scoped to the minimum permissions required.
· Authentication/authorization: privileged acts (mode change, config change, force-close) must be authenticated and authorized.
· Untrusted Telegram input: validate all inbound messages; protect against replay, injection, and rate-limit abuse.
· Dependency and release integrity: pin dependencies; verify release artifacts; maintain supply-chain awareness.
· Storage corruption: detect and recover from storage corruption; maintain backups.
· Backups: what is backed up, how often, and how restoration is tested.
· Controlled startup/shutdown: how the system starts and stops safely without leaving orphan orders or inconsistent state.
· Cross-procedure contradiction check: Before declaring the operations section complete, the design MUST verify that every recovery or restore procedure is compatible with every other procedure that references it. Specifically: a startup sequence that fails a startup check MUST NOT attempt an inline recovery that requires the process to be stopped — an inline recovery that contradicts the shutdown/restore procedure (which requires the process to be stopped) is a contradiction, not a shortcut. The correct behavior when a startup check fails in a way that requires out-of-band recovery is: (a) refuse to start, (b) alert at CRIT class naming the specific check that failed, (c) direct the operator to the out-of-band recovery procedure with a pointer to the section that specifies it, (d) do not attempt the out-of-band procedure inline. Conversely, a recovery procedure that is specified as needing the process stopped MUST NOT be invoked from within the running process under any condition — including as an "attempt recovery" step in a startup or shutdown sequence. The design MUST include a procedural consistency table at the end of the operations section that lists each recovery/restore procedure with its required process state (RUNNING / STOPPED / either), and confirms that no procedure is invoked from a process state that contradicts its requirement.
· Recovery after process death/reboot: what is restored automatically, what requires operator intervention, and how the recovery is verified.
· Logs/metrics/alerts: what is logged, at what level, with what retention, and what triggers an alert.
· Overload behavior: what happens under CPU, memory, disk, network, or API pressure.
· Network interruption: how the system behaves when disconnected, and how it recovers.
· Android process management: how the system handles Doze, App Standby, foreground-service timeouts, OEM process management, and the absence of stable inbound connectivity (§5.2, C15, C16).
· Thermal/resource pressure: how the system behaves under sustained load or thermal throttling (§5.2, C16).
· Incident procedures: what the operator does when something goes wrong, and what the system does in response.

Monitoring must be actionable. An alert that does not lead to a specific action is noise. Separate warnings from fail-closed conditions.

Durable-outage classification. If the design uses severity or outage classes, the classes MUST be defined explicitly in §6 before being cited here. The C12 provisional 3-day outage value governs reconciliation deferral only, and does not constitute a verified exchange retention period. Class-A (C12) is an undefined term and MUST NOT be used as if it were a defined class.

9.9 Telegram as sole interface, not safety authority

The Telegram bot is the sole runtime operator surface, not a safety authority.

· Loss of the bot/API/token/network MUST NOT disable local risk logic or make a pending outcome appear resolved.
· Define what happens to open positions and outstanding orders when the operator is unreachable. A design that assumes the operator is always available is incomplete.
· Compare on-device and off-device deployment only against the owner's allowed infrastructure, resource budget, and operating model. Do not assume a phone is a suitable continuous execution host just because it is the reference device (§3.1 item 8, A8).
· Deployment topology follows the envelope-first derivation (D15, §3.1 item 8), not the phone-vs-cloud default preference.
· The Live/Paper safety interlocks (from §3.1 item 10) are the six correctness gates — data validity, reconciliation, risk-limit definition, credential validity, config integrity, clock offset. The design MUST specify exactly how each interlock is evaluated, what its failure condition is, and what the refusal output looks like. The interlocks are correctness gates, not waiting periods and not profit hurdles.
· Every command's preconditions MUST be evaluable at the time the command is issued, and MUST NOT include checks that require inputs the command does not have. A command that requires evaluating a per-trade risk budget cannot be issued without a specific trade intent (the budget is a function of the trade's notional, stop distance, and equity) — so a general "resume trading" command cannot precondition on per-trade risk; it can precondition on aggregate exposure, position count, drawdown, daily loss, and readiness, but not on a quantity that only exists when an intent exists. A command that requires evaluating clock offset cannot be issued in paper mode (paper does not place real orders, and clock drift only matters when orders could be misdated against a live market) — so a paper-mode command cannot precondition on clock. The design MUST specify, per command, the exact subset of system checks that the command's preconditions cover, and MUST NOT specify "all N checks" as a shorthand when some of the N are not evaluable in that command's context. A precondition list that contains a check the command cannot evaluate is a command-design defect that produces a command that can never pass — which is functionally equivalent to removing the command. The correct rule is: preconditions are the intersection of (checks that the command must enforce) and (checks that the command's context can evaluate).

9.10 Adaptation and model lifecycle

If adaptive weighting, calibration, machine learning, or optimization is included, specify:

· experiment and data versioning — how experiments are tracked, and which data version each experiment used
· training/evaluation boundary — what is trained on what, and what is held out
· parameter registration — the parameter register per §8.6
· selection control — how the winning configuration is selected without multiple-testing bias (§8.4)
· promotion criteria — what must be true for a candidate to become production
· drift and quality monitoring — how drift is detected and what triggers re-evaluation
· rollback — how a promoted configuration is retired
· retirement — how a configuration is permanently removed
· research-to-production separation — live outcomes must not contaminate prior evaluation sets

No live outcome may silently contaminate a prior evaluation set. No self-modifying mechanism may bypass risk limits, authorization, provenance, or validation. Reject autonomous learning if its incremental value does not justify its resource, overfitting, and governance burden. A design with zero autonomous learning is acceptable if requirements justify it.

---

10. Universe selection and resource feasibility

10.1 Universe Derivation Protocol (D14 — E5 treatment)

The trading universe (symbols × timeframes) MUST be derived positively from the following inputs, in this order of priority:

Step 1 — Exchange constraint (A1 + C1):
The universe is bounded by Toobit's listed perpetual-futures symbols and the 14-timeframe capability ceiling in §5.1. 3d is confirmed unsupported. No symbol outside Toobit's perpetual-futures list is admissible absent a material owner authorization. The 14-timeframe list is a ceiling, not a floor. The design's timeframe set is derived in Step 6.

Step 2 — Liquidity screen (C9):
From the Toobit perpetual-futures list, exclude symbols below a liquidity threshold that the model derives from:

· C9 (median 24h notional ≈ 498,844 USDT; 134 symbols ≥ 10M USDT)
· C2 (data depth: 3,500-candle cap may not be reachable for low-liquidity symbols)
· A11 (200–500 USDT capital scale: minimum notional, spread, slippage)
· C6 (taker fee uncertainty 0.05–0.06%)
· C10 (maker-floor cost-per-volatility at 15m+)

The liquidity threshold MUST be explicitly stated, with units, and MUST be derived from the trade-size and cost assumptions — not chosen arbitrarily. The derivation MUST be shown as a calculation chain, not as a conclusion. Specifically, the model MUST display:

1. The trade-size assumption (e.g., notional_per_trade = E × r_trade / (stop_bp / 10,000), expressed parametrically per D17).
2. The book-depth fraction assumption (e.g., "trade must consume ≤ X% of median 24h notional").
3. The inequality that produces the threshold (e.g., notional_per_trade / median_24h_notional ≤ X%).
4. The solved threshold value, with units (in USDT).
5. The chosen X value, with its derivation or DESIGN_JUDGMENT label.

A threshold stated without these five elements is not derived. It is chosen and rationalized retroactively. Relabel per §2 evidence labels. A threshold that is not derivable from the trade-size and cost assumptions is a DESIGN_JUDGMENT that must be flagged.

Step 3 — Data sufficiency screen (C2):
For each surviving symbol, verify the actual available candle depth on Toobit for each candidate timeframe. Symbols whose depth is insufficient for the strategy's lookback requirements are excluded. Depth must be measured or bounded from ratified evidence — not assumed. A symbol whose depth is UNRESOLVED is either excluded with rationale, or included with a PROVISIONAL flag and a resolution path.

Step 4 — Statistical-budget screen (C23, C24):
The admissible number of evaluated configurations per timeframe-cell is bounded (§8.4). For timeframes ≤12h, exhaustive permutation search is inadmissible (C24). The universe size and the strategy's parameter dimensionality must be jointly constrained so that the total evaluated configurations stay within the admissible trial budget. Per §5.4, C23's 10–20 figure is PROVISIONAL pending recomputation; the model MUST NOT use it as a hard ceiling without showing the recomputation.

Step 5 — Mobile compute/storage screen (A8, C14, C16, C19):
The universe must fit within the POCO X6's storage and compute envelope for the phases that must run on-device (per D15 and §3.1 item 8). The model MUST estimate storage from the C19 ratio and compute from measured-or-bounded device capability. C16 (sustained runtime NOT MEASURED) means this screen yields a PROVISIONAL bound pending measurement. The design MUST state the measurement protocol that would resolve the bound.

Step 6 — Strategy-information screen (A6, this §10):
Timeframes not required by the strategy's information needs are excluded, even if listed in C1. Multi-timeframe dependency, if chosen, must be justified per §10.2. Intraday intent (A6) constrains the maximum holding-duration and therefore the relevant timeframe set — but does not exclude timeframes that inform regime, trend, or volatility context.

Step 7 — Pareto frontier:
Steps 2–6 produce constraints. If constraints conflict (e.g., breadth preference A7 vs mobile compute A8), the model MUST derive the Pareto frontier and state the trade-off explicitly. The model MUST NOT silently choose one side. A design that reduces breadth to fit mobile compute must state the breadth reduction as a Pareto choice with its cost. A design that expands to cloud to preserve breadth must state the cloud cost as a Pareto choice with its cost.

Step 8 — Explicit non-use of the 10-symbol measurement:
The 10 symbols in the C19 measurement (§5.3) are not candidates. If any of them survives Steps 2–7, that is incidental. The model MUST NOT use the measurement list as a starting universe.

Step 9 — Universe register:
The final universe MUST be recorded as a register with, per symbol and per timeframe:

Field Content
Symbol Toobit perpetual-futures symbol
Timeframe interval from the capability ceiling
Inclusion reason which of Steps 1–6 admitted it, and on what evidence
Evidence pointer specific record from §5
Verification status PASS / FAIL / BLOCKED / NOT TESTED / PROVISIONAL
Coverage conditions conditions under which coverage widens or narrows
Failure behavior what happens if the symbol or timeframe becomes unavailable

10.2 Timeframe dimensionality (E3.3 treatment)

Timeframe dimensionality is a design decision to be derived, not a default in either direction. The model MUST:

1. Derive from the strategy what information each timeframe would contribute that lower/higher timeframes cannot.
2. Derive from the constraints (statistical budget, compute envelope, storage, latency) what timeframe count is defensible.
3. Derive from the data availability what timeframes are actually usable.
4. Construct the minimal dependency graph that satisfies the strategy's information needs without exceeding the constraints.

The result may be single-timeframe, dual-timeframe, or multi-timeframe — the model MUST NOT bias toward any of these.

If the result is multi-timeframe, the model MUST define, completely:

· which timeframe informs which decision
· over what history
· with what staleness/freshness tolerance
· what happens when a required interval is unavailable
· how the cross-timeframe dependency is estimated and validated
· the causality direction (higher informs lower, lower informs higher, or both)
· the latency implications (higher-timeframe data is typically more lagged)

Cross-instrument features (correlation, SMT divergence, lead-lag, etc.):

Do not use correlation or cross-instrument divergence solely to sound sophisticated. If such a concept is included, state:

· the minimum data coverage it requires (how many symbols, how much history)
· the composition it assumes (which symbols participate)
· the estimation error and how it is quantified
· the failure/degradation policy when the required coverage is unavailable
· the statistical budget it consumes (a correlation feature requires estimating correlation, which uses part of the trial budget)

If the admissible universe cannot support the concept, the model MUST remove it or redesign it explicitly rather than silently returning misleading values. A correlation feature with 3 symbols and 1 month of data is a misleading feature; the model MUST either exclude it or state the limitation explicitly.

10.3 Storage and compute estimation

Calculate or bound storage and processing costs with explicit assumptions. Specifically:

· Storage: estimate per-unit storage from the C19 ratio (§5.3), scaled to the derived universe. Reconcile the stated 1.25 GB anchor only when the proposed data shape matches it. If the data shape differs (adds funding, order-book snapshots, cross-venue data), the ratio does not transfer; a new derivation is required.
· Compute: estimate per-phase compute from the workload characteristics (data volume × operations per unit × parallelism). Compare against measured-or-bounded POCO X6 capability (C14, C16). The comparison yields PROVISIONAL bounds until measured.
· Network: estimate data transfer from API call counts × payload sizes. Compare against the mobile network envelope.

Treat 56 GB free storage as a historical owner snapshot, not a permanent capacity guarantee (§5.2). Current free storage must be re-measured before relying on it.

Separate static specifications from measured runtime behavior. A SoC spec sheet does not tell you how fast the device runs a Python backtest on 3,500 candles × 14 timeframes × N symbols. It tells you the peak theoretical throughput, which is never achieved in practice.

Specify the minimum representative tests needed before claiming the POCO X6 can perform training, backtesting, calibration, Paper, or Live workloads. A test is representative when its data shape, compute profile, and duration match the target workload. A 24-hour charging test (C16) is not representative of a training workload.

Cloud and VPS cost figures (if any) must be stated in USDT or the specific provider currency, with a date and scope, and must not be presented as a fixed commitment. A "modest cloud instance" estimate is DESIGN_JUDGMENT unless a specific provider quote is cited.

---

11. Validation, acceptance, and staged implementation

11.1 Requirement-to-verification matrix

For every critical requirement in the register (§6), provide a validation entry. The matrix MUST be complete per D20 — every field for every requirement, no abbreviations, no "see above," no placeholder rows.

Field	Content
Requirement ID	R-NNN (must exist in the §6 register)
Priority	HARD / SHOULD / MAY
Verification method	DESIGN_INSPECTION / CALCULATION / SOURCE_CHECK / DETERMINISTIC_TEST / EXPERIMENT / OWNER_DECISION
Responsible execution context	who/what executes the verification (model, owner, test harness, external reviewer, etc.)
Required evidence	inputs, fixtures, or conditions the verification needs
Acceptance condition	explicit pass criterion, or UNRESOLVED + method/authority to derive it
Status	PASS / FAIL / BLOCKED / NOT TESTED
Observed result	what was actually observed, with units

Evidence pointer source, calculation, or test result that supports the observed result
Failure response safe fallback, rollback, or refusal behavior
Regression checks which other requirements are affected by a change to this one

A criterion is PASS only when observed evidence satisfies the declared condition. A plan or plausible explanation is not an executed test. BLOCKED means a required dependency, resource, or authorization is unavailable. NOT TESTED means the check was not executed. FAIL means observed evidence violates the criterion.

Do not average hard constraints into a weighted score. A hard-constraint failure cannot be offset by strong performance on unrelated dimensions. A design that scores 90% on SHOULD criteria but fails one HARD criterion is a failing design for the purpose of the HARD criterion.

The verification matrix row count MUST equal the requirement register row count. If the register contains N requirements (R-001 through R-N), the matrix MUST contain exactly N rows — one per requirement, no exceptions. A matrix with fewer rows than the register is a silent omission of the missing requirements: each missing row is a requirement with no verification method, no acceptance condition, no status, and no evidence pointer. A matrix with more rows than the register is a phantom verification of requirements that do not exist. The completeness declaration MUST state both counts explicitly (e.g., "register contains 46 requirements; matrix contains 46 verification rows; R-001 through R-046 inclusive, no gaps, no extras") and MUST NOT rely on "the matrix covers the register" without a count. Any requirement that genuinely does not need a matrix row (e.g., a superseded requirement that was struck from the register) MUST be struck from the register before counting — not omitted from the matrix. The equality is a hard precondition for §11.1 PASS.

11.2 Applicable test selection

Select applicable tests proportionately, choosing a test because it can distinguish acceptable from unacceptable behavior. State each test's coverage and blind spots. Applicable test categories include, without being limited to:

· Mathematical unit tests — formula correctness, unit consistency, edge cases
· Schema/contract tests — interface compatibility between components
· Data-quality and point-in-time tests — look-ahead leakage detection, gap handling, timestamp semantics
· Invariant/property tests — properties that must hold under all inputs (e.g., "risk authority can never be bypassed")
· Deterministic replay — same inputs produce same outputs
· Backtest robustness — sensitivity to cost, slippage, parameter perturbation
· Walk-forward / out-of-sample checks — validation on data not used for calibration
· Execution simulation — order lifecycle including partial fills, cancellations, races
· Fault injection — process death, network loss, storage corruption, clock skew
· API failure and ambiguous-outcome tests — -1007 TIMEOUT handling (C5), rate-limit behavior, bounded retry
· Security/authorization tests — replay, injection, privilege escalation
· Performance/resource tests — CPU, memory, storage, thermal under sustained load
· Mobile process-death/reboot tests — Doze, App Standby, foreground-service timeout, recovery
· Recovery drills — backup restore, reconciliation after crash
· End-to-end tests — full flow from data ingestion to position close

Run a test only when the required tools, inputs, and authorization actually exist. Where execution is unavailable, provide a precise test specification and mark it NOT TESTED or BLOCKED, as applicable.

A test that cannot distinguish a correct implementation from a faulty one is not meaningful validation. State what the test can and cannot establish. A test that passes for both correct and faulty implementations provides no information and should not be listed as validation.

11.3 Mandatory distinctions (do not conflate)

Clearly distinguish seven independent dimensions:

1. Design completeness and consistency — does the design cover all material requirements without contradiction?
2. Source review and evidence status — what was actually read, and what is its scope/status?
3. Research/backtest evidence — what did the historical tests show, under what assumptions?
4. Implementation correctness — does the code match the design? (Usually NOT TESTED in a design engagement.)
5. Paper-operational behavior — did the system behave as designed in paper operation? (Usually NOT TESTED in a design engagement.)
6. Operational readiness — are the operational procedures tested, documented, and rehearsed? (Usually NOT TESTED in a design engagement.)
7. Any future authorization for Live trading — has the owner authorized Live? (Never, in a design engagement.)

A test plan is not an executed test. A simulation is not real execution. A historical result is not a promise of future profit. A passed design review does not mean the system is implemented or safe for real capital.

The seven dimensions must be reported separately in the final status record. Combining them into a single "ready" claim is a D21 violation (it substitutes a summary for the actual seven-dimensional status).

11.4 Staged implementation order

Derive implementation order from dependencies and risk reduction, not an arbitrary number of phases. Specifically:

· Dependency order: a component that others depend on is implemented first.
· Risk-reduction order: a component whose failure is most consequential is implemented first (even if it has no dependents).
· Verification order: a component whose verification unblocks other verifications is implemented first.

If a future repository blueprint is justified, propose only the level of structure supported by the design. Do not create boilerplate files, placeholder modules, or one folder per concept. A repository structure that mirrors the design's natural decomposition is acceptable. A repository structure that anticipates future features is a P1 violation (quantitative completeness pressure in repository form).

The future repository blueprint, implementation sequence, production code, and the owner's knowledge repository are distinct things. Do not conflate them. Do not modify or create files in any external repository as part of this engagement.

---

12. Deliverables and final response format

12.1 Structure from requirements (D12 — E2 treatment)

The structure of the deliverable (number of sections, documents, diagrams, tables) MUST be derived from the requirements and the design's natural decomposition.

Before writing any deliverable, the model MUST:

1. List the material design questions that must be answered.
2. Group them by natural cohesion (same subsystem, same decision, same evidence type).
3. Derive the minimum structure that answers all material questions without repetition or fragmentation.
4. Write the deliverable in that structure.
5. Then audit against the §12.2 checklist.

A design that requires 8 sections is acceptable. A design that requires 40 sections because the checklist has 40 items is not acceptable. The model MUST be able to justify every section's existence on the basis of a material design question it answers.

12.2 Coverage checklist (omission-audit tool, not structure)

The following coverage checklist is an omission-audit tool, not a structure template. After the design is written in its natural structure, the model MUST audit it against each item and record one of: COVERED, INTENTIONALLY EXCLUDED + reason, NOT RELEVANT + reason, or UNRESOLVED + resolution path.

The checklist (not a required structure):

· executive summary, objective, and scope
· requirements/constraint registry, assumptions, conflicts, and owner decisions
· evidence/source register and the persisted pre-research design checkpoint
· considered alternatives and architecture rationale
· data universe, lineage, and quality rules
· statistical research and trial-budget methodology
· strategy/decision/portfolio behavior
· independent risk policy and its non-delegable owner boundary
· execution lifecycle, state reconciliation, and failure behavior
· Telegram command/permission/alert contract
· deployment/resource feasibility per phase
· security, reliability, recovery, and operations
· interfaces and state ownership
· verification/acceptance matrix and adversarial scenarios
· implementation roadmap and future repository blueprint if justified
· traceability from requirements to design decisions and tests
· concise evidence/decision record for consequential choices, including the strongest considered alternative and why it was rejected
· capability and execution record stating which sources, tools, calculations, tests, and independent reviews were actually used, which were unavailable, and what was not performed
· acceptance-status summary (PASS / FAIL / BLOCKED / NOT TESTED) for critical criteria
· remaining uncertainties, limitations, and the next highest-value validation action, if any

How to use the checklist:

1. Write the design in its natural structure first.
2. Audit against each item.

3. Record each item as COVERED, INTENTIONALLY EXCLUDED + reason, NOT RELEVANT + reason, or UNRESOLVED + resolution path.
4. Never restructure the design to match the checklist. The checklist adapts to the design, not the reverse.
5. A design that covers 12 of 20 checklist items with justification is acceptable and preferable to a design that mechanically covers all 20.

The number of checklist items is a property of this prompt's construction, not a completeness target.

12.3 The completeness declaration (D20 mandate)

Every deliverable MUST satisfy §2.2.2 (D20). A deliverable included per D12 must be presented completely. An item excluded per D12 must be excluded with rationale, not summarized.

The final deliverable MUST include the completeness declaration per §2.2.5. The declaration is itself a material item and MUST satisfy D20/D21/D22.

12.4 Language and format

The principal design specification must be written in professional English Markdown.

Owner-facing progress/status notes and the short final summary must be in Persian.

If the environment can create files, provide a verified link to the actual created artifact. If it cannot, provide the complete content in manageable parts, numbered per §2.2.4 (D22).

Do not claim to have generated a file, reviewed a source, or run a test unless it happened.

All monetary quantities MUST be denominated in USDT (or the specific quote currency of the instrument) — never in USD. If a USD approximation is presented for context, it must be labeled as a derived approximation with the conversion assumption shown.

12.5 Multi-message delivery

If the complete deliverable exceeds a single-message limit, deliver it across multiple sequential messages per §2.2.4 (D22). Specifically:

· Plan the message count in the first message.
· Number each part (Part 1/N, Part 2/N, ..., Part N/N).
· Deliver each part completely for its declared scope.
· Include a completeness declaration in each part.
· Cross-reference between parts.
· Deliver the final part with the overall completeness declaration.

Do not truncate. Do not summarize. Do not defer. Do not abbreviate. Length is a delivery logistics problem, not a content completeness problem.

---

13. Final adversarial self-audit

Before declaring the design complete, verify all of the following. Each audit item is a material requirement and MUST satisfy D20 (present with rationale, status, and consequence).

13.1 Constraints and objectives

□ Every hard owner constraint is traceable and honored, or its conflict is explicitly reported.
□ Preferences and brainstorms were not silently converted to requirements (per D13 and §3.2).
□ The objective is measurable where possible; unresolved risk appetite and success criteria are not invented (§3.3, D17).
□ The Live/Paper mode request is reconciled with essential safety interlocks without arbitrary profitability or elapsed-time gates (§3.1 item 10, D16, §9.9).
□ The sole Telegram interface remains an interface requirement, not a trust assumption, and not a process-topology constraint (§3.1 item 2).
□ All monetary quantities are in USDT, not USD (§9.6, §12.4).

13.2 Evidence integrity

□ Sources actually opened/read are distinguished from filenames, snippets, owner summaries, and inaccessible resources.
□ All claims preserve scope, date/freshness, and status (§2, §4.1).
□ Contradictions — including the mark-price change semantics conflict (C13) — are recorded rather than silently resolved.
□ The 149-record summary (C25) is not falsely described as 149 fully passing tests (§5.5).
□ No re-investigation was performed without a material, authorized reason; no unresolved finding was presented as settled.
□ Every material claim traced to an evidence pointer or an explicit label satisfies §2.1.0c's four-category

test.
□ No claim is presented as fact that is not traceable to one of the four evidence-first categories (§2.1.0c).
□ Plausible-sounding content is not presented as evidence (no hedged fabrications — §2.1.0c).

13.3 Quantitative integrity

□ No look-ahead, leakage, multiple testing, or unrealistic execution assumptions are concealed (§8).
□ Trial budget calculations are unit-consistent and respect the ratified constraint against exhaustive full-grid search at ≤12h (C24, §5.4, §8.4).
□ C23's 10–20 and thousands figures are labeled PROVISIONAL pending recomputation wherever used.
□ Predictive quality, backtest return, net economic value, paper results, and future Live results remain distinct (§8.3, §11.3).
□ The 200–500 USDT planning case is used without pretending it is actual current equity or proof of profitability (§3.1 item 11, D17).
□ All cost, risk, and quantity calculations state units and assumptions (§9.6, D17).
□ No preemptive acceptance of any strategy is presented as an established result (§8.0).

13.4 Safety, architecture, and implementation

□ Risk authority is independent and cannot be bypassed by an optimizer or interface failure (§9.6).
□ Unknown/stale risk, data, and exchange state fail safely (§9.5, §9.6).
□ Timeouts, partial fills, retry behavior, and reconciliation have explicit state transitions (§9.7).
□ Resource requirements are estimates until measured; the POCO X6's workload limits are not claimed as tested (§5.2, §10.3).
□ Every component, dependency, and technology choice has a justified purpose (D18, §9.2).
□ Every critical requirement has a verification method and testable acceptance rule or an explicit unresolved procedure (§11.1).
□ Design, implementation, validation, and Live authorization are not conflated (§11.3).
□ All feasible deployment topologies are enumerated before compromise (Step 4b, §3.1 item 8).

13.5 Capability use and quality-gate fidelity

□ The actual tool/access envelope was identified without guessing hidden settings or claiming unavailable capabilities (§2.1.1).
□ High-impact choices were evaluated against criteria declared before scoring; hard constraints were not traded away for a higher aggregate score (§2.1.3, §11.1).
□ Additional research, model calls, tests, and iterations each had a material purpose; redundant activity did not substitute for useful verification (§2.1.3).
□ Every critical acceptance status is supported by observed evidence or truthfully marked FAIL, BLOCKED, or NOT TESTED (§2.1.4).
□ The adversarial review attempted to falsify the design, and material findings were resolved, contained, or disclosed with consequences (§2.1.5).
□ Revisions were followed by relevant regression checks where feasible; the final report does not overstate the validation performed.
□ Maximum capability was used as a mandate, not a preference (§2.1 opening).

13.6 Completeness and delivery (D20/D21/D22)

□ Every material item satisfies D20 (content, rationale, status, evidence pointer, consequence) — §2.2.2.
□ No D21-prohibited pattern appears in the output — §2.2.3.
□ All parts delivered completely per D22 — §2.2.4.
□ The completeness declaration exists and is complete — §2.2.5.
□ Self-verification V-1 through V-12 was performed — §2.2.6.
□ All artifacts listed in the artifact registry are actually delivered per §2.2.8.
□ No form of content reduction (compression, simplification, summarization, abbreviation, omission, referral, superficiality, deferral, truncation) was used at any stage — §2.2.1.

13.7 Prompt fidelity

□ No undefined requirement IDs, change classes, or abbreviations are used as if already defined (§6, §5.0, §5.1 Class-A caveat).
□ Every C-series, A-series, D-series, E-series, P-series, G-series, T-C-series, V-series identifier used anywhere in the output is defined in §5.0 or inline where it first appears.
□ No contradictory source-policy rule remains (§4).
□ The initial checkpoint is called pre-research, not falsely "uncontaminated" (§4.3).
□ The deliverable's size and structure serve implementation and review rather than impressiveness (§12.1, D12).
□ P1 (quantitative completeness pressure) is neutralized: section counts, audit counts, and deliverable checklist counts are not treated as targets (§2.1.0, §12.2).
□ P2 (linear ordering pressure) is neutralized: gate names, step numbers, and section order are reference labels, not execution mandates (§2.1.2, D11).
□ P3 (negative-anchor pressure) is neutralized: "do not do X" is read as "X is not predetermined," not as "X is a candidate to reject" (§2.1.0b, D13).
□ P4 (mobile-as-axis pressure) is neutralized: POCO X6 is a feasibility constraint, not an architecture driver (§3.1 item 8, D15).
□ P5 (statistical-rigor-as-axis pressure) is neutralized: rigor validates strategies, it does not

preemptively reject them (§8.0, D19).
□ All USD references have been converted to USDT (§9.6, §12.4, §13.1).
□ All deviations from owner-intended phase vocabulary are disclosed in a single completeness table (§3.1 item 10, A5 patch).
□ All programming-language, framework, runtime, and library decisions are treated as delegated (§3.1 item 12) and are not paused for per §2.1.7.

Correct defects where possible. Otherwise record each material limitation, its risk, the decision needed, and the next admissible method for resolving it.

---

14. Final instruction

Deliver the most defensible design the available inputs, permitted evidence, and actual tools allow. Be willing to reject the owner's suggested mechanisms and familiar industry patterns where they do not earn their place. Be equally willing to retain a simple method when it best satisfies the requirements. The architecture, analytical methods, optimizer, deployment topology, trading universe, component boundaries, document count, and repository structure must emerge from the verified problem — not from a template, not from owner vocabulary, not from industry convention, not from any numeric artifact in this prompt.

Remain within this design-only engagement. Do not implement, deploy, access the account, transmit secrets, place orders, or claim guaranteed profitability. State clearly what has been established, what is a reasoned proposal, what remains unresolved, and what must be measured or authorized before a future implementation or Live operation.

Use the capability-maximization protocol in §2.1 throughout the engagement. Demonstrate its execution through the quality of the decisions, the provenance of the evidence, the validity of checks, and the honesty of the final status record — not by claiming that you "used maximum effort." The target is the strongest defensible result under actual constraints, not an unverifiable claim of perfection.

Use the completeness mandate in §2.2 throughout the engagement. Demonstrate it through full-content delivery, explicit status labels, evidence pointers, and the completeness declaration — not by claiming completeness. A design that says "complete" without the declaration violates D20. A design with the declaration and honest UNRESOLVED/BLOCKED items is complete.

Honor the dependency-first ordering (D11) and positive derivation (D13) throughout the engagement. A design that follows the numbered sequence without dependency analysis violates D11. A design that reads the negative anchors as exclusion mandates violates D13. The sequence is a reference; the dependency is the mandate. The negative anchor is a pressure-removal; the positive derivation is the mandate.

Honor the evidence-first mandate (§2.1.0c). Every assertion, claim, calculation, and decision MUST satisfy one of the four categories in §2.1.0c. Plausible-sounding content without evidence is a fabrication. "I do not know" is a legitimate output when the evidence is absent; it is not a failure, it is a disclosure.

Identifier integrity. If the design introduces identifiers that are used in more than one place — alert codes, error codes, state-transition names, requirement IDs, evidence IDs, subsystem names, config-key classes — each identifier MUST appear in exactly one catalog section where it is fully defined (content, trigger or derivation, status, consequence). Every subsequent use of the identifier MUST point to that catalog, not redefine it in place. An identifier that is used in a downstream section without being defined in a catalog is a dangling identifier and is a D20 violation of the same severity as a status-only entry. The completeness declaration MUST enumerate every catalog and confirm: (a) the catalog is complete (no identifier used elsewhere is missing from the catalog), and (b) no identifier is defined in two catalogs. Example: if the design defines alert code A-NETWORK-LOSS in its alert catalog, then a later operations section may cite A-NETWORK-LOSS but must not define a variant or introduce a related A-NETWORK-LOSS-EXTENDED outside the catalog — the extended variant must be added to the catalog before the citation.

Honor the incremental persistence mandate (§2.2.8). Deliver artifacts as they are produced, not accumulated for a single final delivery. Update the artifact registry with each phase. A design that produces excellent artifacts lost to a mid-process interruption has produced zero artifacts.

Honor the pause-and-present protocol (§2.1.7). When a pre-undeclared owner-level decision meets the five pause conditions, stop and present. When a delegated decision is encountered, do not pause — derive and proceed. The distinction between owner-level and delegated decisions is defined in §2.1.7.

All monetary quantities are USDT. Never USD. When a USD context is unavoidable, label the conversion and state the assumption.
